ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 9. МЕТОДЫ ОЦЕНКИ ПЛОТНОСТИ ДЛЯ ВЫЯВЛЕНИЯ АНОМАЛИЙ В КИБЕРБЕЗОПАСНОСТИ

План лекции:

1. Общая идея методов оценки плотности.
2. Kernel Density Estimation (KDE).
3. Gaussian Mixture Model (GMM).
4. Histogram-Based Outlier Score (HBOS).

Методы оценки плотности для выявления аномалий в кибербезопасности

1. Общая идея методов оценки плотности

Методы оценки плотности основаны на предположении: нормальные события образуют плотные области в пространстве признаков, а аномальные события располагаются в разреженных областях.

Пусть сетевое событие описывается вектором:

$$\mathbf{x} = (x_1, x_2, \ldots, x_i, \ldots, x_d),$$

где признаками x_i могут быть:

- количество соединений в минуту;
- число уникальных адресатов;
- объём исходящего трафика;
- средний размер пакета;
- продолжительность соединения.

Пусть $f(\mathbf{x})$ — оценка плотности данных в точке $\mathbf{x}$. Она показывает насколько типично такое событие в наших данных. Если $f(\mathbf{x})$ большое — событие обычное, лежит в «густом скоплении» нормальных сессий. Если $f(\mathbf{x})$ маленькое — оно находится в области, где нормальных событий почти не бывает.

Тогда естественной оценкой аномальности события является величина:

$$S(\mathbf{x}) = -\log f(\mathbf{x}).$$

Чем меньше плотность $f(\mathbf{x})$, тем больше значение $S(\mathbf{x})$ и тем более аномальным считается событие.

Почему именно логарифм и знак минус.

1. Инверсия и удобство шкалы.

Плотность $f(\mathbf{x})$ лежит в диапазоне $[0, \infty)$ (для непрерывного распределения), и чем она меньше, тем более аномально событие. Но для детектора удобнее, когда «больше = хуже». Минус решает эту проблему:

- если значение $f(\mathbf{x})$ очень маленькое (аномалия), то $\log f(\mathbf{x})$ сильно отрицательный, а $-\log f(\mathbf{x})$ становится большим положительным;

- если значение $f(\mathbf{x})$ большое (норма), то $-\log f(\mathbf{x})$ будет иметь маленькое значение.

2. Логарифм делает числа удобными для практики.

В реальных данных плотности могут быть очень маленькими (например, 10^{-12}). Работать с такими числами неудобно: они плохо читаются, плохо визуализируются. Логарифм переводит экспоненциально малые значения в читаемый диапазон:

- $f(\mathbf{x}) = 10^{-12} \Rightarrow \log_{10} f(x) = -12$, тогда $S(\mathbf{x}) = 12$.
- $ff(\mathbf{x}) = 10^{-6} \Rightarrow S(\mathbf{x}) = 6$.

Это уже удобные числа, которые можно использовать как «баллы риска» и ставить пороги (например, алерт при $S(\mathbf{x}) > 8$).

3. Математическое удобство при работе с многомерными данными.

Если признаки условно независимы, то плотность раскладывается в произведение:

$$f(\mathbf{x}) = f_1(x_1) \cdot f_2(x_2) \cdots f_d(x_d).$$

Тогда логарифм превращает произведение в сумму:

$$S(\mathbf{x}) = -\log f(\mathbf{x}) = -\sum_{i=1}^{d} \log f_i(x_i) = \sum_{i=1}^{d} s_i(x_i),$$

где $s_i(x_i) = -\log f_i(x_i)$ — вклад каждого признака в аномальность.

После этого можно оценивать вклад отдельных признаков: например, понять, что аномальность выросла из-за «слишком большого объёма исходящего трафика», а не из-за числа портов.

В задачах кибербезопасности для выявления аномалий используются следующие методы оценки плотности:

- Histogram-Based Outlier Score (HBOS);
- Kernel Density Estimation (KDE);
- Gaussian Mixture Model (GMM).

Не все рассматриваемые методы строят явную вероятностную плотность:

- KDE и GMM непосредственно оценивают $f(\mathbf{x})$;
- HBOS приближает совместную плотность через одномерные гистограммы.

Методы оценки плотности могут применяться в двух разных режимах.

1. Обучение без учителя: Outlier Detection

Алгоритм получает набор данных, в котором предположительно содержатся и нормальные события, и небольшое количество атак. Метки событий модели неизвестны.

Задача состоит в том, чтобы найти редкие наблюдения внутри этой же выборки.

Схема:

$$\text{смешанная выборка} \rightarrow \text{оценка плотности} \rightarrow \text{поиск редких объектов}.$$

Основное предположение: аномалий значительно меньше, чем нормальных событий.

Опасность состоит в том, что несколько похожих атак могут образовать собственную плотную группу. Тогда алгоритм может принять её за один из режимов нормального поведения.

2. Обучение только на нормальных данных: Novelty Detection

Имеется историческая выборка, содержащая только подтверждённые нормальные события. На ней строится профиль нормы. Затем модель применяется к новым событиям.

Схема:

$$\text{нормальные события} \rightarrow \text{модель нормы} \rightarrow \text{оценка новых событий}.$$

Метод обнаруживает не выбросы в обучающей выборке, а новые события, не соответствующие известному профилю. Такая постановка также называется обнаружением новизны.

Главная опасность — загрязнение обучающей выборки незамеченными атаками и неполное описание нормальных режимов работы.

2. Подготовка исходных данных

Для изучения методов будет использоваться набор данных, содержащий агрегированные характеристики сетевых сессий организации.

Каждая сессия содержит три числовых признака:

Признак	Описание	Возможный смысл аномалии
`connections_per_min`	Количество соединений в минуту	Сканирование, автоматизированная активность
`unique_destinations`	Количество уникальных узлов назначения	Сканирование или боковое перемещение
`outbound_mb`	Объём исходящего трафика, МБ	Возможная эксфильтрация данных

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

In [ ]:
```
# @title
rng = np.random.default_rng(42)

FEATURES = [
    "connections_per_min",
    "unique_destinations",
    "outbound_mb"
]


def make_normal_group(
    n,
    mean,
    covariance,
    traffic_type,
    dataset_part
):
    """Создаёт один режим нормального сетевого поведения."""

    values = rng.multivariate_normal(
        mean=mean,
        cov=covariance,
        size=n
    )

    # Признаки не могут быть отрицательными.
    values = np.clip(
        values,
        a_min=[0.1, 0.1, 0.1],
        a_max=None
    )

    result = pd.DataFrame(
        values,
        columns=FEATURES
    )

    result["traffic_type"] = traffic_type
    result["scenario"] = "normal"
    result["is_attack"] = 0
    result["dataset_part"] = dataset_part

    return result


# Ковариационные матрицы задают разброс и взаимосвязи
# признаков внутри каждого нормального режима.
web_covariance = np.array([
    [64, 15, 20],
    [15,  9,  5],
    [20,  5, 49]
])

api_covariance = np.array([
    [225, 5, 30],
    [  5, 2.25, 2],
    [ 30, 2, 64]
```

```
])

backup_covariance = np.array([
    [9, 1,   5],
    [1, 0.36, 1],
    [5, 1, 625]
])

# Исторические подтверждённые нормальные данные.
reference_groups = [
    make_normal_group(
        360,
        mean=[55, 14, 25],
        covariance=web_covariance,
        traffic_type="web",
        dataset_part="reference_normal"
    ),
    make_normal_group(
        180,
        mean=[135, 6, 35],
        covariance=api_covariance,
        traffic_type="api",
        dataset_part="reference_normal"
    ),
    make_normal_group(
        60,
        mean=[18, 2, 220],
        covariance=backup_covariance,
        traffic_type="backup",
        dataset_part="reference_normal"
    )
]

# Нормальная часть мониторингового набора.
monitoring_normal_groups = [
    make_normal_group(
        144,
        mean=[55, 14, 25],
        covariance=web_covariance,
        traffic_type="web",
        dataset_part="monitoring"
    ),
    make_normal_group(
        72,
        mean=[135, 6, 35],
        covariance=api_covariance,
        traffic_type="api",
        dataset_part="monitoring"
    ),
    make_normal_group(
        24,
        mean=[18, 2, 220],
        covariance=backup_covariance,
        traffic_type="backup",
        dataset_part="monitoring"
    )
]


def make_attack_group(n, mean, std, scenario):
```

```
    """Создаёт группу искусственных атак."""

    values = rng.normal(
        loc=mean,
        scale=std,
        size=(n, len(FEATURES))
    )

    values = np.clip(values, 0.1, None)

    result = pd.DataFrame(
        values,
        columns=FEATURES
    )

    result["traffic_type"] = "unknown"
    result["scenario"] = scenario
    result["is_attack"] = 1
    result["dataset_part"] = "monitoring"

    return result


attack_groups = [
    make_attack_group(
        4,
        mean=[220, 150, 5],
        std=[8, 10, 1],
        scenario="port_scan"
    ),
    make_attack_group(
        3,
        mean=[55, 13, 650],
        std=[3, 2, 30],
        scenario="exfiltration"
    ),
    make_attack_group(
        4,
        mean=[75, 1, 3],
        std=[3, 0.2, 0.5],
        scenario="beaconing"
    ),
    make_attack_group(
        3,
        mean=[145, 45, 20],
        std=[5, 3, 3],
        scenario="lateral_movement"
    )
]

df = pd.concat(
    reference_groups
    + monitoring_normal_groups
    + attack_groups,
    ignore_index=True
)

df.insert(
    0,
    "session_id",
```

```
    [f"S{i:04d}" for i in range(len(df))]
)

df[FEATURES] = df[FEATURES].round(2)
```

In [ ]:
```
df.head()
```

Out[ ]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	sce
0	S0000	49.93	10.93	28.74	web	n
1	S0001	41.26	13.53	30.24	web	n
2	S0002	53.06	13.54	25.97	web	n
3	S0003	64.45	14.59	24.73	web	n
4	S0004	58.31	13.76	19.01	web	n

В набор данных добавлены служебные столбцы, которые необходимы для разделения данных с целью демонстрации различных режимов работы и для проверки качества выявления аномалий:

- `dataset_part`:
  - `dataset_part == monitoring`: мониторинговая часть данных, которая будет использоваться в режиме обучения без учителя (содержит как нормальные сессии, так и атаки);
  - `dataset_part == reference_normal`: часть данных, которая будет использоваться в режиме обучения только на нормальных данных (содержит только нормальные сессии);
- `traffic_type`: истинный режим нормального трафика:
  - `web`: обычная работа пользователей с веб-ресурсами;
  - `api` — интенсивные обращения приложений к небольшому числу API;
  - `backup` — небольшое число соединений с большим объёмом данных;
- `scenario`: название сценария для наблюдение:
  - `normal`: нормальная сессия;
  - `port_scan`: большое количество соединений с множеством узлов;
  - `exfiltration`: аномально большой объём исходящего трафика;
  - `beaconing`: периодическая связь с одним адресатом небольшими пакетами;
  - `lateral_movement`: взаимодействие с нетипично большим числом внутренних узлов;
- `is_attack`: истинная метка нормы или атаки.

Эти метки не поступают на вход алгоритмов выявления аномалий.

Структура набора данных:

```
df
├── reference_normal — 600 подтверждённых нормальных сессий
└── monitoring       — 254 новые сессии
```

```
├── 240 нормальных сессий
└── 14 атак
    ├── 4 port_scan
    ├── 4 beaconing
    ├── 3 exfiltration
    └── 3 lateral_movement
```

Разделение данных.

In [ ]:
```python
# Исторически нормальные данные
reference_df = df[df["dataset_part"] == "reference_normal"].copy()

# Мониторинговые данные
monitoring_df = df[df["dataset_part"] == "monitoring"].copy()
```

В режиме обучения только на нормальных данных будет использоваться историческая нормальная часть. На ней модель изучает нормальное поведение. Затем на данных мониторинговой части модель проверяет новые события.

В режиме обучения без учителя будет использоваться мониторинговая часть. Алгоритм не знает, какие события являются атаками, и ищет выбросы внутри всей мониторинговой выборки.

Визуализируем распределение данных.

Три признака невозможно полностью показать на обычном двумерном графике, поэтому построим две проекции.

In [ ]:
```python
scenario_colors = {"normal": "steelblue",
                   "port_scan": "red",
                   "exfiltration": "darkorange",
                   "beaconing": "purple",
                   "lateral_movement": "green"}

fig, axes = plt.subplots(1, 2, figsize = (8, 6))

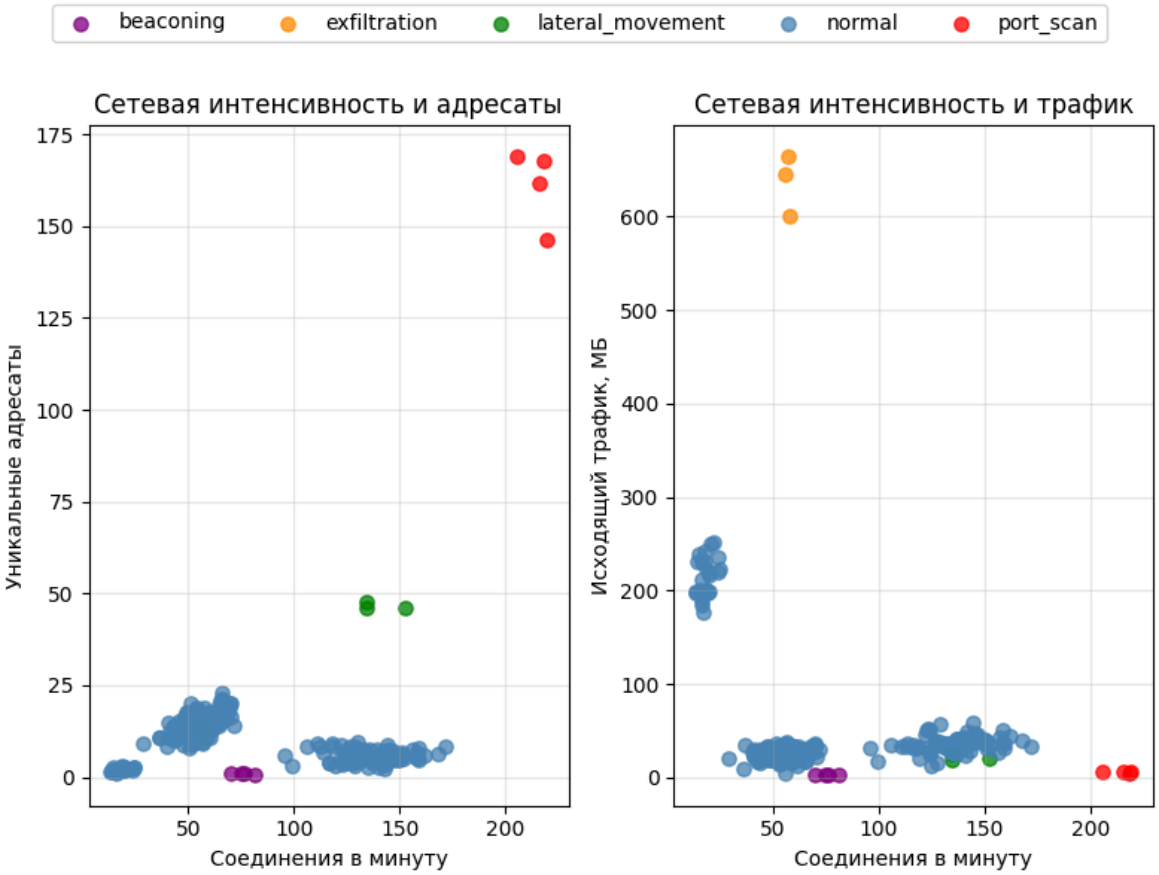
for scenario, group in monitoring_df.groupby("scenario"):
    axes[0].scatter(group["connections_per_min"],
                    group["unique_destinations"],
                    color = scenario_colors[scenario],
                    label = scenario, alpha = 0.75, s = 45)

    axes[1].scatter(group["connections_per_min"],
                    group["outbound_mb"],
                    color = scenario_colors[scenario],
                    label = scenario, alpha = 0.75, s = 45)

axes[0].set_xlabel("Соединения в минуту")
axes[0].set_ylabel("Уникальные адресаты")
axes[0].set_title("Сетевая интенсивность и адресаты")
axes[0].grid(alpha = 0.3)

axes[1].set_xlabel("Соединения в минуту")
axes[1].set_ylabel("Исходящий трафик, МБ")
axes[1].set_title("Сетевая интенсивность и трафик")
axes[1].grid(alpha=0.3)
```

```
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc = "upper center", ncols = 5)
plt.tight_layout(rect=[0, 0, 1, 0.92])
```



Полученные графики подтверждают применимость методов оценки плотности: нормальные события формируют плотные кластеры, аномальные — лежат в разреженных областях.

Вспомогательные функции

Создадим две вспомогательные функции, которые будут формировать результаты работы методов.

In [ ]:
```
# Функция для расчёта и вывода метрик качества детекции.

from sklearn.metrics import confusion_matrix
def print_detection_result(method_name, y_true, y_pred):
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()

    print(method_name)
    print(f"TP — обнаруженные атаки: {tp}")
    print(f"FP — ложные тревоги: {fp}")
    print(f"FN — пропущенные атаки: {fn}")
    print(f"TN — корректно распознанная норма: {tn}")
```

In [ ]:
```
# Функция визуализации результатов выявления аномалий

def plot_detection_result(data, prediction, title):
    prediction = np.asarray(prediction, dtype = bool)
```

```
actual_attack = (data["is_attack"].to_numpy().astype(bool))

fig, axes = plt.subplots(1, 2, figsize = (14, 6))

projections = [("connections_per_min",
                "unique_destinations",
                "Соединения в минуту",
                "Уникальные адресаты"),
               ("connections_per_min",
                "outbound_mb",
                "Соединения в минуту",
                "Исходящий трафик, МБ")]

for ax, (x_column, y_column, x_label, y_label
) in zip(axes, projections):

    # Объекты, которые модель считает нормальными
    ax.scatter(data.loc[~prediction, x_column],
               data.loc[~prediction, y_column],
               color = "steelblue", alpha = 0.65,
               s = 35, label = "Модель: норма")

    # Объекты, которые модель считает аномальными
    ax.scatter(data.loc[prediction, x_column],
               data.loc[prediction, y_column],
               color = "red", s = 65,
               label = "Модель: аномалия", zorder = 3)

    # Истинные атаки показываются только
    # для учебной проверки результатов
    ax.scatter(data.loc[actual_attack, x_column],
               data.loc[actual_attack, y_column],
               facecolors = "none",
               edgecolors = "black",
               linewidths = 1.5, s = 120,
               label = "Внедрённая атака", zorder = 4)

    ax.set_xlabel(x_label)
    ax.set_ylabel(y_label)
    ax.grid(alpha = 0.3)

axes[0].set_title("Проекция 1")
axes[1].set_title("Проекция 2")

# Получаем элементы легенды только с первого графика,
# чтобы не дублировать одинаковые обозначения.
handles, labels = (axes[0].get_legend_handles_labels())

# Общий заголовок располагается сверху.
fig.suptitle(title, fontsize = 16, fontweight = "bold", y = 0.98)

# Легенда располагается ниже общего заголовка.
fig.legend(handles, labels, loc = "upper center",
           bbox_to_anchor = (0.5, 0.91), ncols = 3, frameon = True)

# Верхняя граница 0.82 оставляет пространство
# для общего заголовка и легенды.
plt.tight_layout(rect = [0, 0, 1, 0.82])
```

3. Kernel Density Estimation (KDE)

Алгоритм метода

KDE — это непараметрический метод оценки функции плотности вероятности (PDF). В отличие от параметрических методов (где заранее предполагается форма распределения, например, нормальное, и подбираются параметры), KDE не накладывает жёстких ограничений. Он строит гладкую кривую, «накладывая» на каждую точку данных небольшое «пятно» (ядро). Общая плотность получается усреднением всех ядер.

Пусть есть набор данных из n объектов: $X = x_1, \dots, x_i, \dots x_n$. Каждый объект описывается d признаками. Для каждого объекта $x_i = (x_{i1}, x_{i2}, \dots, x_{ij}, \dots, x_{id})$, где x_{ij} — значение j-го признака у i-го объекта.

Цель: построить оценку плотности $\hat{f}(x)$ в произвольной точке x, чтобы затем использовать её для детекции аномалий:

низкая плотность ⇒ точка редкая ⇒ потенциальная аномалия.

Шаг 1. Выбор функции ядра.

Ядро — это симметричная функция, которая определяет, насколько сильно обучающее наблюдение x_i влияет на оценку плотности в анализируемой точке x.

Для KDE могут применяться различные ядра:

- гауссовское;
- равномерное;
- треугольное;
- ядро Епанечникова;
- экспоненциальное.

Наиболее часто используется гауссовское ядро:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right),$$

где u - нормированное расстояние по одному признаку.

Шаг 2. Выбор ширины ядра (полосы пропускания - bandwidth) h_j для каждого признака.

Величина $h_j > 0$ управляет «шириной» влияния точек обучающей выборки x_i на анализируемую точку x по j-му признаку.

- При увеличении h_j в оценке плотности участвуют более далёкие соседи.
- При уменьшении h вклад имеют только близкие точки.

Существует несколько способов выбора h_j. Рассмотрим один из них.

Правило Сильвермана (для каждого признака отдельно):

$$h_j = \left(\frac{4}{(d+2)n} \right)^{\frac{1}{d+4}} \cdot \sigma_j,$$

где:

- h_j — полоса пропускания для j-го признака;
- n — число обучающих точек;
- d — количество признаков;
- σ_j — стандартное отклонение j-го признака (на обучающей выборке).

Шаг 3. Подготовка нового события.

Пусть необходимо оценить новое сетевое событие:

$$x_q = (x_1, x_2, \cdots, x_j, \cdots, x_d).$$

Шаг 4. Расчёт нормированного различия по каждому признаку.

По каждому признаку j для нового объекта x_q и обучающего объекта x_{ij} рассчитывается:

$$u_{qi} = \frac{x_{qj} - x_{ij}}{h},$$

где:

- u_{qi} — нормированное расстояние от нового объекта x_q до объекта обучающих данных x_i по j-му признаку;
- x_{qj} — значение j-го признака нового объекта x_q;
- x_{ij} — значение j-го признака i-го обучающего объекта;
- h — ширина ядра.

Каждое j-е значение показывает сходство нового и обучающего объектов по одному признаку.

Шаг 5. Расчёт ядра по каждому признаку.

Каждое нормированное расстояние подставляется в выбранное на первом шаге ядро (в нашем примере - гауссовское ядро):

$$k(u_{qi}) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u_{qi}^2}{2} \right),$$

где:

- $k(u_{qi})$ — вклад i-го обучающего объекта по j-му признаку в оценку плотности;
- u_{qi} — нормированное расстояние между новым и обучающим объектами.

Шаг 6. Произведение ядер по всем признакам.

Для каждого i-го обучающего объекта значения ядер перемножаются:

$$K(x_i) = \prod_{j=1}^{d} k(u_{qi}),$$

где:

- $K(x_i)$ — совместный многомерный вклад i-го обучающего объекта в оценку плотности;
- $k(u_{qi})$ — отдельный вклад i-го обучающего объекта по j-му признаку в оценку плотности;
- d — количество признаков.

Если новый объект близок к обучающему по всем признакам, произведение будет сравнительно большим.

Ядро $k(u_{qi})$ — неотрицательная функция с максимумом при $u_{qi} = 0$

- Если новый объект близок к i-му обучающему по всем признакам:
  $\forall j \; |u_{qi}| \approx 0 \Rightarrow k(u_{qi}) \approx k(0) \approx \max.$
- Если хотя бы по одному признаку $|u_{qi}|$ велико - $|u| \to \infty$:
  $\exp\left(-\frac{u^2}{2}\right) \to 0 \Rightarrow k(u) \to 0$. Тогда всё произведение $K(x_i) \approx 0$ из-за умножения на близкое к нулю значение.

По сути, значения вычисленных ядер показывают для каждой обучающей точки насколько она «похожа» на новую точку.

Шаг 7. Учёт ширины многомерного ядра.

Полный вклад i-го обучающего объекта:

$$C(x_i) = \frac{1}{h^d} \cdot K(x_i) \text{ или } C(x_i) = \frac{1}{h^d} \prod_{j=1}^{d} k\left(\frac{x_{qj} - x_{ij}}{h}\right).$$

Здесь:

- $C(x_i)$ — полный вклад i-го обучающего объекта в оценку плотности;
- $K(x_i)$ — произведение ядер по всем признакам i-го обучающего объекта;
- h — ширина ядра;
- d — количество признаков;
- h^d — поправка на многомерный масштаб.

В результате получаем полные вклады каждого i-го обучающего объекта.

Шаг 8. Суммирование и усреднение вкладов.

Вклады всех обучающих объектов складываются и делятся на их количество:

$$\hat{f}_h(x_q) = \frac{1}{n}\sum_{i=1}^{n} C(x_i),$$

где:

- $\hat{f}_h(x_q)$ — KDE-оценка плотности в новой точке x_q;
- n — количество обучающих объектов;
- $C(x_i)$ — вклад i-го обучающего объекта в оценку плотности;
- $\sum_{i=1}^{n}$ — суммирование по всем обучающим объектам.

Подставляя формулу для $C(x_i)$, получаем полную KDE:

$$\hat{f}_h(x_q) = \frac{1}{nh^d} \cdot \sum_{i=1}^{n} \prod_{j=1}^{d} k\left(\frac{x_{qj} - x_{ij}}{h}\right).$$

Интерпретация:

- высокая плотность — анализируемый объект x_q находится рядом со многими обучающими объектами x_i;
- низкая плотность — объект x_q находится в разреженной области.

Если в эту формулу подставить выражение для выбранного ядра, то можно получить ее развернутую запись. Например для гауссовского ядра:

$$\hat{f}(x_q) = \frac{1}{nh^d} \sum_{i=1}^{n} \prod_{j=1}^{d} \left[\frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x_{qj} - x_{ij}}{h}\right)^2\right)\right].$$

Шаг 9. Расчёт оценки аномальности.

Плотность преобразуется в оценку аномальности:

$$S_{\mathrm{KDE}}(x_q) = -\log \hat{f}_h(x_q),$$

где:

- $S_{\mathrm{KDE}}(x_q)$ — оценка аномальности нового объекта q;
- $\hat{f}_h(x_q)$ — KDE-плотность нового объекта q.

Интерпретация:

- небольшое значение $S_{\mathrm{KDE}}(x_q)$ — объект находится в плотной области;
- большое значение $S_{\mathrm{KDE}}(x_q)$ — объект находится в разреженной области.

Шаг 10. Выбор порога.

При обучении только на нормальных данных порог выбирается по независимой нормальной калибровочной выборке:

$$T = Q_{1-\alpha}\left(S_{\mathrm{KDE}}\left(X_{\mathrm{calibration}}\right)\right),$$

где:

- T — порог аномальности;
- $Q_{1-\alpha}$ — квантиль уровня $1-\alpha$;

- α — допустимая доля ложных предупреждений;
- $X_{\text{calibration}}$ — калибровочные нормальные события;
- S_{KDE} — оценки аномальности объектов колибровочной выборки.

Шаг 11. Формирование итоговой метки.

Новое событие x_q признаётся аномальным, если:

$$S_{\text{KDE}}(x_q) > T.$$

Итоговая метка:

$$\hat{y}(x_q) = \begin{cases} 1, & S_{\text{KDE}}(x_q) > T; \\ 0, & S_{\text{KDE}}(x_q) \leq T. \end{cases}$$

Здесь:

- $\hat{y}(x_q)$ — предсказанная метка;
- $S_{\text{KDE}}(x_q)$ — оценка аномальности;
- T — порог;
- 1 — аномалия;
- 0 — нормальное событие.

Реализация KDE в sklearn

В `sklearn` метод KDE реализован классом: `from sklearn.neighbors import KernelDensity`.

Основные параметры:

- `bandwidth` (по умолчанию: 1.0): задаёт ширину окна h (гладкость оценки плотности). Может принимать значения `float` или `scott`, `silverman`, которые позволяют выбрать правила формирования окна;
- `kernel` (по умолчанию `gaussian`): вид ядра;
- `metric` (по умолчанию `euclidean`): определяет метрику расстояния, по которой считается близость точек;
- `algorithm` (по умолчанию `auto`): вид алгоритма перебора точек обучающей выборки. Варианты: `auto`, `ball_tree`, `kd_tree`, `brute`.

`KernelDensity` является оценщиком плотности, а не готовым детектором аномалий.

Поэтому он:

- не формирует метки аномалий автоматически;
- не предоставляет отдельный метод `predict()` для формирования меток класса «норма/аномалия».

Основные методы:

- `fit()` : «запоминает» обучающие данные и строит вспомогательную структуру для быстрого вычисления расстояний между объектами;
- `score_samples()` : возвращает логарифм плотности каждого объекта.

Порог и метки необходимо формировать самостоятельно. Согласно документации `sklearn` , метод `score_samples(X)` возвращает логарифм плотности для каждого переданного объекта. Чем больше это значение, тем выше плотность.
Документация

Для удобства будем использовать оценку аномальности:

$$S_{KDE}(x) = -log\hat{f}_h(x).$$

Тогда при малых значениях $S_{KDE}(x)$ объект находится в области высокой плотности, при больших - в разреженной области.

Реализация при обучении без учителя

In [ ]:
```
df.head()
```

Out[ ]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	sce
0	S0000	49.93	10.93	28.74	web	n
1	S0001	41.26	13.53	30.24	web	n
2	S0002	53.06	13.54	25.97	web	n
3	S0003	64.45	14.59	24.73	web	n
4	S0004	58.31	13.76	19.01	web	n

Функция визуализации KDE-оценок.

Получает уже рассчитанные оценки аномальности и показывает их распределение отдельно для известных нормальных событий и атак.

In [ ]:
```
def plot_kde_scores(data, scores, threshold, title):
    """
    Параметры:
    data — DataFrame с событиями и известными метками is_attack:
           0 — нормальное событие, 1 — атака.
    scores — одномерная последовательность оценок аномальности:
             score = -log(оценённой плотности).
             Чем больше оценка, тем более необычно событие.
    threshold — ранее выбранный порог:
                событие считается аномальным при score > threshold.
    title — заголовок графика.

    Порядок оценок в scores должен совпадать с порядком строк data.
    """
```

```
    # Преобразуем оценки в массив NumPy.
    # Это позволяет выбирать оценки с помощью логических масок.
    scores = np.asarray(scores)

    # Маска известных нормальных событий:
    # True стоит на позициях строк, где is_attack равен 0.
    normal_mask = (data["is_attack"].to_numpy() == 0)

    # Маска известных атак:
    # True стоит на позициях строк, где is_attack равен 1.
    attack_mask = (data["is_attack"].to_numpy() == 1)

    plt.figure(figsize = (8, 5))

    # Выбираем оценки известных нормальных событий
    # и строим гистограмму их распределения.
    plt.hist(scores[normal_mask], bins = 30, alpha = 0.65,
             color = "steelblue", label = "Нормальные события")

    # На том же рисунке строим распределение оценок известных атак.
    plt.hist(scores[attack_mask], bins = 20, alpha = 0.75,
             color = "red", label = "Атаки")

    # Проводим вертикальную линию в точке выбранного порога.
    plt.axvline(threshold, color = "black", linestyle = "--",
                linewidth = 2, label = f"Порог = {threshold:.3f}")

    # Горизонтальная ось показывает оценки аномальности.
    # Это отрицательный логарифм плотности, а не вероятность атаки.
    plt.xlabel("KDE anomaly score = -log(плотность)")

    # Вертикальная ось показывает количество событий
    # в каждом интервале гистограммы.
    plt.ylabel("Количество событий")

    plt.title(title)
    plt.legend()
    plt.grid(alpha = 0.3)
    plt.tight_layout()
```

Разделим датасет на две части:

- исторические подтверждённые нормальные данные;
- смешаннные мониторинговые данные (нормальные данные + атаки).

In [ ]:
```
# Исторические подтверждённые нормальные данные
reference_df = df.loc[df["dataset_part"] == "reference_normal"].copy()

# Новые мониторинговые события:
# 240 нормальных событий и 14 внедрённых атак
monitoring_df = df.loc[df["dataset_part"] == "monitoring"].copy()
```

В режиме обучения без учителя модель получает смешанную мониторинговую выборку $X_{unsupervised}$. При этом KDE не знает, какие события являются атаками.

Схема работы:

смешанная выборка → масштабирование → KDE → оценка плотности этих же объектов → выбор редких событий.

Подготовка признаков.

In [ ]:
```
# Берём только числовые признаки.
X_unsupervised_raw = (monitoring_df[FEATURES].to_numpy())
X_unsupervised_raw[:5]
```

Out[ ]:
```
array([[56.5 , 15.11, 22.97],
       [57.21, 19.07, 26.54],
       [51.52, 13.02, 22.68],
       [40.72, 14.97, 30.16],
       [53.06, 13.11, 23.7 ]])
```

Масштабирование признаков.

In [ ]:
```
unsupervised_scaler = StandardScaler()

# В режиме без учителя масштабирование
# строится на всей анализируемой выборке.
X_unsupervised = (unsupervised_scaler.fit_transform(X_unsupervised_raw))
X_unsupervised[:5]
```

Out[ ]:
```
array([[-0.47948747,  0.09505378, -0.35035616],
       [-0.46380287,  0.29426388, -0.30828921],
       [-0.58950058, -0.01008488, -0.35377337],
       [-0.82808324,  0.088011  , -0.26563308],
       [-0.55548046, -0.00555738, -0.34175424]])
```

Создание и обучение KDE.

In [ ]:
```
from sklearn.neighbors import KernelDensity
kde_unsupervised = KernelDensity(
    # Гауссовское ядро
    kernel = "gaussian",

    # Ширина ядра после стандартизации
    bandwidth = 0.35,

    # Евклидово расстояние
    metric = "euclidean",

    # Автоматический выбор KDTree или BallTree
    algorithm = "auto"
)

# Все мониторинговые события участвуют
# в построении оценки плотности.
kde_unsupervised.fit(X_unsupervised)
```

Out[ ]:
```
KernelDensity(bandwidth=0.35)
```

Получение логарифма плотности.

Для каждого объекта получаем: $L(x) = log\left[\hat{f}_h(x)\right]$.

Большое значение $L(x)$ означает высокую плотность.

In [ ]:
```
unsupervised_log_density = (kde_unsupervised.score_samples(X_unsupervised))
unsupervised_log_density[:5]
```

Out[ ]:
```
array([-0.38064962, -0.57442436, -0.39931834, -0.7189826 , -0.38388038])
```

Преобразование в оценку аномальности.

Для удобства меняем знак логарифма плотности и получаем оценку аномальности:

$$S_{KDE}(x) = -L(x) = log\hat{f}_h(x).$$

Теперь:

- маленькое значение — объект похож на другие;
- большое значение — объект находится в области низкой плотности.

In [ ]:
```
unsupervised_scores = (-unsupervised_log_density)
```

Выбор порога принятия решений.

В режиме без учителя калибровочной нормальной выборки нет. Поэтому необходимо задать предполагаемую долю аномалий.

Для нашего набора данных мы знаем точную долю атак (среднее столбца `is_attack`) - примерно 0,55.

В реальных системах доля атак неизвестна. Она определяется:

- экспертной оценкой;
- допустимым числом предупреждений;
- результатами предыдущих периодов;
- небольшой размеченной выборкой.

Предполагаем, что около 5,5% событий могут быть аномальными.

Порог: $T = Q_{1-\alpha}(S_{KDE}(x))$, где α — предполагаемая доля аномалий.

In [ ]:
```
# Предполагаемая доля аномалий.
expected_anomaly_fraction = 0.055

unsupervised_threshold = np.quantile(unsupervised_scores,
                                     1 - expected_anomaly_fraction)

print(f'Порог KDE без учителя: {round(unsupervised_threshold,4)}')
```

```
Порог KDE без учителя: 2.6596
```

Формирование меток аномальности:

- 1 — аномалия;

- 0 — нормальное событие.

In [ ]:
```
unsupervised_prediction = (unsupervised_scores > unsupervised_threshold).astype(
unsupervised_prediction[:5]
```

Out[ ]:
```
array([0, 0, 0, 0, 0])
```

Запись результатов в исходную таблицу.

In [ ]:
```
monitoring_df["kde_unsupervised_log_density"] = unsupervised_log_density
monitoring_df["kde_unsupervised_score"] = unsupervised_scores
monitoring_df["kde_unsupervised_anomaly"] = unsupervised_prediction
```

Просмотр событий с наибольшим KDE score.

In [ ]:
```
monitoring_df[["session_id",
               *FEATURES,
               "scenario",
               "kde_unsupervised_score",
               "kde_unsupervised_anomaly"]].sort_values("kde_unsupervised_score"
                                                        ascending = False).head(
```

Out[ ]:

	session_id	connections_per_min	unique_destinations	outbound_mb	scena
842	S0842	219.44	146.29	6.08	port_sc
845	S0845	58.14	11.27	601.34	exfiltrat
846	S0846	56.84	14.26	664.41	exfiltrat
853	S0853	152.45	46.08	20.14	lateral_movem
844	S0844	55.73	13.29	644.75	exfiltrat
841	S0841	205.60	168.93	5.68	port_sc
843	S0843	215.66	161.48	5.54	port_sc
840	S0840	218.28	167.82	4.79	port_sc
852	S0852	134.50	47.52	24.31	lateral_movem
851	S0851	134.57	46.17	18.86	lateral_movem
849	S0849	81.06	0.65	3.25	beacon
837	S0837	16.96	2.25	177.34	norr
820	S0820	22.19	2.42	251.45	norr
788	S0788	99.23	3.27	17.06	norr
847	S0847	76.22	0.94	3.39	beacon
821	S0821	20.39	1.91	249.70	norr
848	S0848	74.86	0.90	2.84	beacon
751	S0751	171.95	8.40	33.59	norr
815	S0815	95.58	5.81	30.80	norr
839	S0839	16.14	0.91	184.11	norr

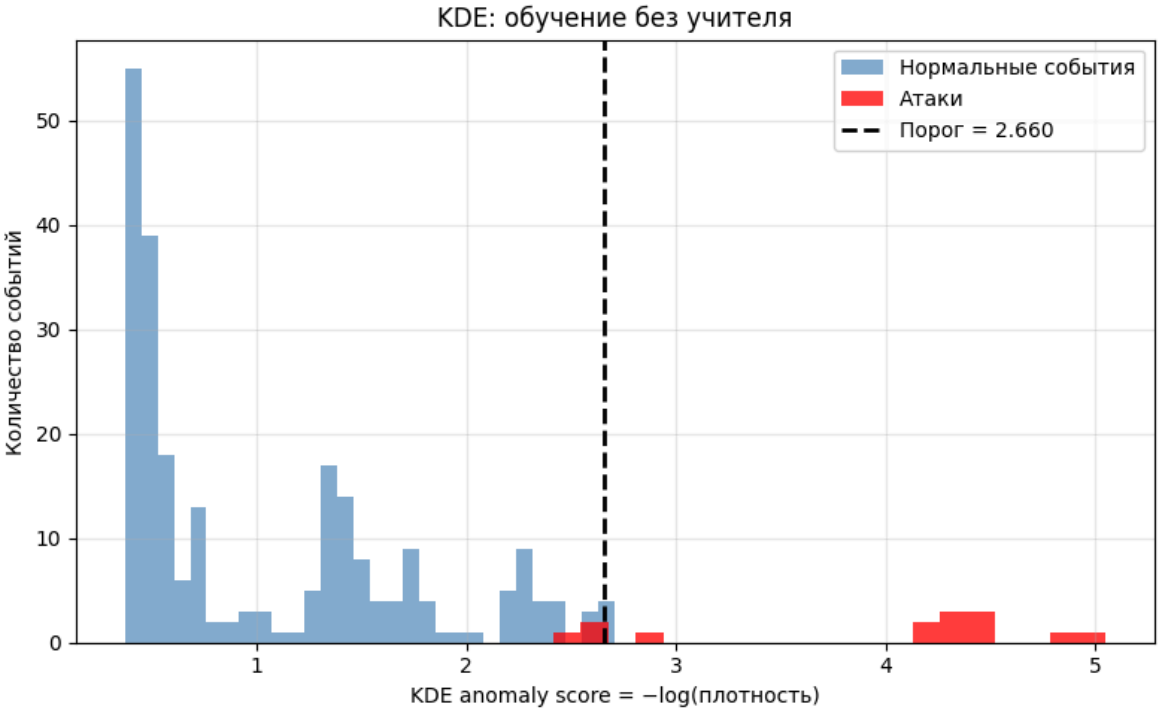
Оценивание качества детекции аномалий.

In [ ]:
```
print_detection_result("KDE: обучение без учителя",
                       monitoring_df["is_attack"],
                       unsupervised_prediction)
```

```
KDE: обучение без учителя
TP — обнаруженные атаки: 11
FP — ложные тревоги: 3
FN — пропущенные атаки: 3
TN — корректно распознанная норма: 237
```

Визуализация результатов детекции аномалий.

In [ ]:
```
plot_kde_scores(monitoring_df,
                unsupervised_scores,
                unsupervised_threshold,
                "KDE: обучение без учителя")
```



Реализация при обучении только на нормальных данных

В этом режиме модель обучается только на подтверждённых нормальных событиях X_{normal}.

Нормальная выборка разделяется на две части:

- normal_fit_df — для обучения KDE;
- normal_calibration_df — колибровочная для выбора порога аномальности.

После этого модель применяется к новым мониторинговым событиям.

Схема:

- нормальные fit-данные → KDE нормы;
- нормальные calibration-данные → порог аномальности → метки;
- мониторинговые данные → оценки качества детекции.

Калибровочная выборка решает три критические задачи, без которых детекция аномалий в кибербезопасности становится ненадёжной:

1. Избежать «подгонки» порога принятия решений под обучающие данные. Если подбирать порог по тем же данным, на которых модель обучалась (или вообще не разделять), модель будет слишком оптимистично оценивать свои границы. Калибровочная выборка — это «честный» тест профиля нормы: мы смотрим, насколько сильно даже легитимные, но редкие нормальные события могут отклоняться от среднего поведения.
2. Сделать порог интерпретируемым и контролируемым. Используя калибровочную выборку, мы явно говорим: «допускается 1% ложных срабатываний на нормальных данных» (через 99-й процентиль).

3. Защитить от скрытых аномалий в «нормальных» данных. Даже в датасете, который считается «нормальным», могут быть редкие легитимные всплески или единичные атаки. Разделение обучающую и калибровочные выборки помогает:

- обучить профиль на большей части данных;
- проверить, насколько «чистой» выглядит оставшаяся часть.

Если на калибровочной выборке уже много «очень подозрительных» наблюдений, это сигнал, что в «норме» есть проблемы. Это особенно важно в кибербезопасности: если случайно обучить профиль на данных, где уже есть атаки, граница нормы сдвинется, и реальные атаки потом могут выглядеть «нормально».

Разделение нормальных данных на обучающую и колибровочные выборки.

In [ ]:
```
normal_fit_df, normal_calibration_df = train_test_split(
    reference_df,

    # 75% — обучение KDE,
    # 25% — калибровка порога
    test_size = 0.25,

    random_state = 42,

    # Сохраняем соотношение web/API/backup
    # в обеих выборках.
    stratify = reference_df["traffic_type"])
```

Масштабирование признаков.

Принципиальным отличием от режима обучения без учителя является обучение `scaler` только на нормальных данных - атаки не должны влиять на параметры масштабирования.

In [ ]:
```
normal_scaler = StandardScaler()

# fit_transform выполняется только
# на обучающей нормальной части.
X_normal_fit = (normal_scaler.fit_transform(normal_fit_df[FEATURES]))

# Калибровочные данные не участвуют
# в вычислении среднего и std.
X_normal_calibration = (normal_scaler.transform(normal_calibration_df[FEATURES]))

# Мониторинговые данные также преобразуются
# с параметрами нормальной обучающей части.
X_monitoring = (normal_scaler.transform(monitoring_df[FEATURES]))
```

Создание и обучение KDE.

В построении плотности не участвуют:

- калибровочные события;
- мониторинговые нормальные события;
- атаки.

Модель строит только профиль нормального поведения.

In [ ]:
```
kde_normal = KernelDensity(kernel = "gaussian",
                           bandwidth = 0.35,
                           metric = "euclidean",
                           algorithm = "auto")

# Обучение на нормальных данных
kde_normal.fit(X_normal_fit)
```

Out[ ]:
```
KernelDensity(bandwidth=0.35)
```

Оценка калибровочных событий.

Калибровочные события:

- являются нормальными;
- не участвовали в обучении;
- имитируют новые нормальные сетевые сессии.

In [ ]:
```
calibration_log_density = (kde_normal.score_samples(X_normal_calibration))
calibration_scores = (-calibration_log_density)
```

Выбор порога аномальности.

Допустим, предполагается, что около 1% новых нормальных событий могут оказаться выше порога:

$$T = Q_{1-\alpha}(S_{calibration}), \text{ где } \alpha = 0,01.$$

Это не означает, что в калибровочной выборке содержится 1% атак - она считается нормальной. Значение 0.01 задаёт допустимую долю крайних нормальных оценок.

In [ ]:
```
target_false_alarm_rate = 0.01
normal_threshold = np.quantile(calibration_scores,
                               1 - target_false_alarm_rate)

print(f'Калибровочный порог KDE: {round(normal_threshold,4)}')
```

```
Калибровочный порог KDE: 3.8019
```

Оценивание мониторинговых событий.

Нужны для оценивания качества детекции.

In [ ]:
```
monitoring_log_density_normal = (kde_normal.score_samples(X_monitoring))
monitoring_scores_normal = (-monitoring_log_density_normal)
```

Формирование меток аномальности.

Нужны для оценивания качества детекции.

In [ ]:
```
normal_prediction = (monitoring_scores_normal > normal_threshold).astype(int)
```

Запись результатов в исходную таблицу.

In [ ]:
```
monitoring_df["kde_normal_log_density"] = monitoring_log_density_normal
monitoring_df["kde_normal_score"] = monitoring_scores_normal
monitoring_df["kde_normal_anomaly"] = normal_prediction
```

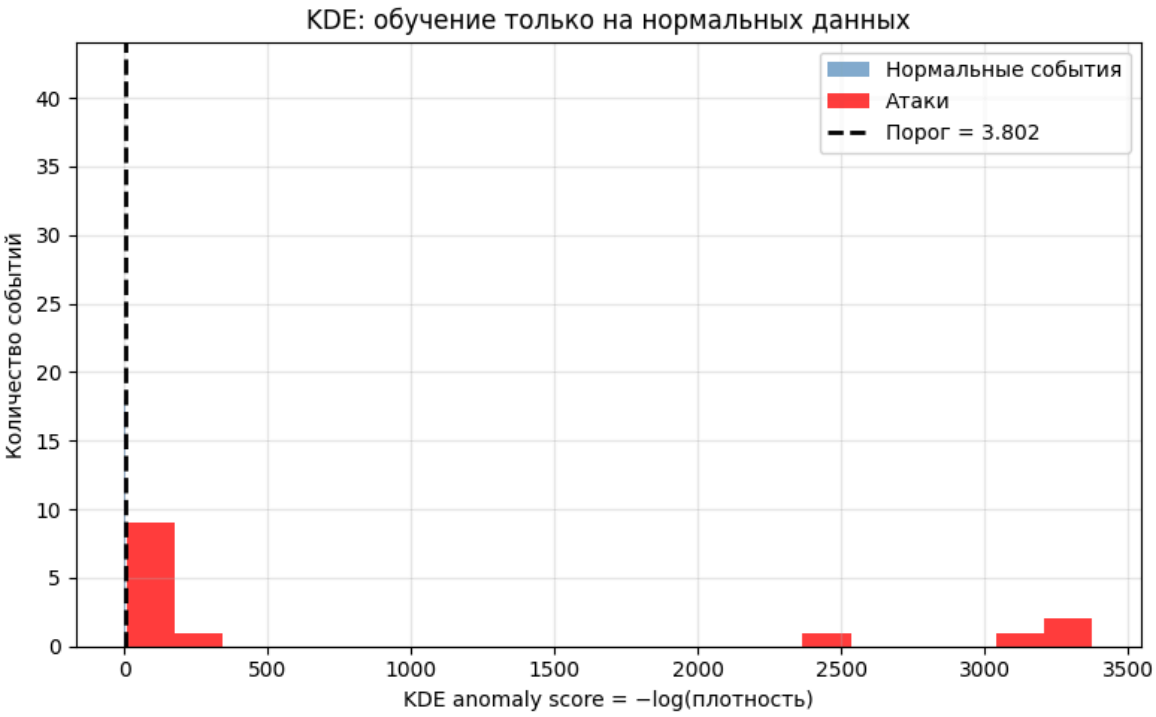
Оценивание качества детекции аномалий.

In [ ]:
```
print_detection_result("KDE: обучение только на нормальных данных",
                       monitoring_df["is_attack"],
                       normal_prediction)
```

```
KDE: обучение только на нормальных данных
TP — обнаруженные атаки: 14
FP — ложные тревоги: 1
FN — пропущенные атаки: 0
TN — корректно распознанная норма: 239
```

Визуализация результатов детекции аномалий.

In [ ]:
```
plot_kde_scores(monitoring_df,
                monitoring_scores_normal,
                normal_threshold,
                "KDE: обучение только на нормальных данных")
```



Сравнение результатов детекции аномалий

Различие в результатах KDE при разных режимах обучения связано с тем, как модель «видит» данные: включает ли она атаки в построение плотности или опирается только на нормальное поведение. Это критично для задач

кибербезопасности — от выбора стратегии зависит, насколько надёжно будут детектироваться аномалии.

1. Обучение без учителя.

В этом случае модель строит оценку плотности по всей выборке — и по нормальным сессиям, и по атакам. Формально плотность записывается так:

$$\hat{f}(\mathbf{x}) = \frac{1}{n}\left[\sum_{\text{normal}} K(\mathbf{x}-\mathbf{x}_i) + \sum_{\text{attack}} K(\mathbf{x}-\mathbf{x}_i)\right].$$

Здесь:

- n — общее число объектов;
- $K(\cdot)$ — ядро (например, гауссовское);
- суммы идут по нормальным событиям и по атакам соответственно.

Атаки вносят в оценку плотности свои собственные «ядра»: вокруг каждой атакующей точки создаётся локальный «холм» плотности. Если несколько атак близки друг к другу (например, сканирование портов с похожими параметрами), они взаимно повышают плотность друг друга. В итоге такие атаки перестают быть «редкими» — их плотность оказывается не очень низкой, и они хуже выделяются как аномалии.

Кроме того, атаки влияют на масштабирование. Если перед KDE применяется `StandardScaler`, то среднее и стандартное отклонение считаются по всем данным, включая атаки. Из-за этого:

- нормальные точки могут «сдвигаться» относительно своих естественных режимов;
- разброс нормальных значений искусственно увеличивается, и плотность нормы становится более «размазанной».

В результате атаки, которые в чистом профиле нормы были бы ярко выраженными аномалиями, в такой модели могут выглядеть почти как норма.

2. Обучение только на нормальных данных.

Здесь модель обучается исключительно на подтверждённо легитимном трафике. Плотность строится только по нормальным объектам:

$$\hat{f}_{\text{normal}}(\mathbf{x}) = \frac{1}{n_{\text{normal}}} \sum_{\text{normal}} K(\mathbf{x}-\mathbf{x}_i),$$

где n_{normal} — число нормальных объектов.

При таком подходе у атак нет собственных ядер в модели: они не участвуют в формировании плотности. Если новое событие (в том числе атака) сильно отличается от нормальных режимов, оно попадает в область низкой плотности:

$$\hat{f}_{\text{normal}}(\mathbf{x}_{\text{attack}}) \approx 0.$$

Тогда логарифмическая мера аномальности даёт большое значение:

$$-\log \hat{f}_{\text{normal}}(\mathbf{x}_{\text{attack}}) \gg 0.$$

Именно это и нужно для детекции: чем дальше событие от нормального профиля, тем выше его скор аномальности.

Практические рекомендации.

Режим обучения без учителя подходит, когда:

- отсутствует проверенная нормальная история;
- необходимо исследовать уже собранный массив журналов;
- предполагается, что атаки редки;
- допустимо задавать ожидаемую долю аномалий.

Режим обучения только на нормальных данных предпочтителен, когда:

- имеется надёжная история нормальной работы;
- новые события поступают после обучения;
- требуется непрерывный мониторинг;
- необходимо обнаруживать новые режимы поведения.

Но этот режим требует, чтобы нормальная история:

- не содержала скрытых атак;
- охватывала веб-доступ, API, резервное копирование и другие законные режимы;
- регулярно обновлялась с учётом дрейфа поведения.

Важно понимать, что `KernelDensity` реализует только непосредственно оценивание плотности. Превращение этой плотности в детектор аномалий — выбор знака оценки, порога и правила классификации — выполняется разработчиком.

Ограничения метода KDE

Kernel Density Estimation является гибким непараметрическим методом, но его качество существенно зависит от данных, числа признаков, ширины ядра и способа выбора порога.

1. Высокая чувствительность к ширине ядра.

Если h слишком мало:

- около каждого обучающего объекта возникает отдельный узкий пик;
- плотность становится неустойчивой;
- небольшие отклонения получают низкую плотность;
- увеличивается количество ложных тревог;
- модель фактически запоминает обучающую выборку.

Например, немного необычная, но законная сессия администратора может быть признана атакой.

Если h слишком велико:

- плотность чрезмерно сглаживается;
- разные режимы нормального поведения сливаются;
- разреженные области между нормальными группами получают завышенную плотность;
- реальные атаки могут быть пропущены.

Выбор h необходимо выполнять на отдельных проверочных данных или с помощью кросс-валидации.

2. Проклятие размерности.

При росте размерности (увеличении количества признаков) расстояния между объектами начинают становиться похожими, а оценка плотности — ненадёжной.

Возможные решения:

- отбор информативных признаков;
- удаление дублирующих признаков;
- PCA;
- автоэнкодер для снижения размерности;
- построение отдельных KDE-моделей для разных групп признаков.

3. Высокая вычислительная стоимость.

Для оценки плотности нового объекта требуется определить его расстояние до большого количества обучающих объектов.

Для миллионов сетевых событий это может быть слишком дорого.

Возможные решения:

- уменьшение обучающей выборки;
- сокращение числа признаков;
- применение более быстрых методов, например HBOS или Isolation Forest.

4. Необходимость хранения обучающих объектов.

KDE не сводит данные к небольшому числу параметров. Для вычисления плотности требуется хранить обучающие наблюдения или структуру дерева, построенную на них.

При большой обучающей выборке KDE требует значительного объёма памяти.

В кибербезопасности это особенно важно, поскольку сетевые сенсоры и SIEM могут получать миллионы событий в сутки.

7. Одна ширина ядра может быть недостаточна.

В стандартной реализации часто используется одно значение h для всех областей пространства и всех признаков.

Но нормальная активность может иметь области с разной плотностью:

- веб-трафик образует большую плотную группу;
- API — другую группу;
- резервное копирование — небольшую разреженную группу;
- административные операции встречаются редко.

При единой ширине окна возникает противоречие:

- малое h подходит для плотного веб-кластера, но создаёт ложные тревоги для разреженного backup-кластера;
- большое h подходит для backup-кластера, но может скрывать аномалии около веб-трафика.

Возможные решения:

- адаптивное окно;
- отдельные модели по типам устройств;
- отдельные модели для пользователей и ролей;
- локальные методы, например LOF;
- GMM с несколькими компонентами.

8. Атаки влияют на модель в режиме без учителя.

При обучении без учителя KDE строится на смешанных данных. Каждая атака создаёт вокруг себя собственное ядро. Поэтому её плотность автоматически увеличивается.

Если несколько атак похожи друг на друга, они дополнительно повышают плотность друг друга.

Этот эффект особенно выражен, если:

- атаки многочисленны;
- атаки похожи друг на друга;
- объект оценивается на той же выборке, на которой KDE обучалась.

9. Требование чистой нормальной выборки.

Если в исторической выборке присутствуют незамеченные атаки, KDE сформирует около них области плотности и может принять подобное поведение за нормальное.

Поэтому обучающая выборка должна:

- проходить предварительную проверку;
- охватывать достаточно длительный период;

- анализироваться экспертами;
- очищаться от известных инцидентов.

10. Необходимость ручного выбора порога.

`KernelDensity` возвращает плотность, но не определяет, какое значение считать аномальным. Порог приходится выбирать самостоятельно.

Порог может определяться:

- по предполагаемой доле аномалий;
- по квантилю нормальной калибровочной выборки;
- по допустимому числу ложных тревог;
- по небольшой размеченной выборке.

11. KDE-плотность не является вероятностью атаки.

Значение KDE является оценкой плотности данных, а не вероятностью атаки. Низкая плотность означает только, что событие редко относительно обучающей выборки.

Редкое событие может быть:

- атакой;
- ошибкой сбора;
- новым законным процессом;
- действием администратора;
- техническим сбоем.

Поэтому KDE должен формировать сигнал для дальнейшей проверки, а не окончательный вердикт.

12. Ограниченная интерпретируемость.

KDE показывает, что объект находится в области низкой плотности, но не всегда объясняет:

- какой признак вызвал тревогу;
- какое сочетание признаков оказалось необычным;
- к какому типу атаки относится событие;
- какие действия должен предпринять аналитик.

Для объяснения приходится дополнительно анализировать:

- расстояния по признакам;
- ближайшие нормальные объекты;
- отклонения от медиан;
- контекст пользователя и устройства.

4. Gaussian Mixture Model (GMM)

Gaussian Mixture Model, GMM — модель, предполагающая, что данные сформированы смесью нескольких многомерных нормальных распределений.

Каждое распределение называется:

- компонентом смеси;
- гауссовским компонентом;
- скрытым режимом данных.

В задачах кибербезопасности разные компоненты могут описывать разные режимы типичного нормального поведения системы:

- веб-доступ пользователей;
- обращения приложений к API;
- резервное копирование;
- административные операции.

Общая плотность определяется как взвешенная сумма плотностей компонентов:

$$p(x) = \sum_{k=1}^{K} \pi_k \cdot \mathcal{N}\left(x \mid \mu_k, \Sigma_k\right).$$

Здесь:

- $p(x)$ — плотность смеси в точке x;
- x — анализируемый объект;
- K — количество гауссовских компонентов;
- π_k — вес k-го компонента (доля объектов, относящихся к k-му компоненту);
- μ_k — вектор средних k-го компонента;
- Σ_k — ковариационная матрица k-го компонента;
- $\mathcal{N}(x \mid \mu_k, \Sigma_k)$ — многомерная нормальная плотность;
- $\sum_{k=1}^{K}$ — суммирование по всем компонентам.

Веса компонентов удовлетворяют условиям: $\pi_k \geq 0$ и $\sum_{k=1}^{K} \pi_k = 1$.

Алгоритм метода

Пусть обучающая выборка содержит n объектов: $X = \{x_1, x_2, \ldots, x_i, \ldots, x_n\}$.

Каждый объект описывается d признаками: $x_i = \{x_{i1}, x_{i2}, \ldots, x_{ij}, \ldots, x_{id}\}$.

Шаг 1. Выбор количества компонентов.

В задачах кибербезопасности каждый компонент — это отдельный «режим» нормального поведения системы.

Например:

- Компонент 1: «ночной трафик с малым числом коротких сессий».
- Компонент 2: «дневной HTTP-трафик с большими сессиями».

- Компонент 3: «сканирование портов: много очень коротких сессий, почти без данных».

То есть компонент — это отдельная статистическая модель, которая описывает одну группу точек в полном пространстве признаков.

В учебном датасете нормальная активность содержит три режима:

- `web`;
- `api`;
- `backup`.

Поэтому ожидается: $K = 3$.

Однако в реальной задаче количество компонентов заранее неизвестно. Его можно выбирать с помощью различных способов.

Одним из наиболее распространенных является информационный критерий BIC — Bayesian Information Criterion:

$$BIC = -2 \log L + s \cdot \log n,$$

где:

- L — максимальное правдоподобие модели;
- s — количество свободных параметров модели;
- n — количество обучающих объектов.

Это выражение характеризует «соревнование» двух сил:

- $-2 \log L$ — плата за то, что модель плохо описывает данные. Чем лучше модель описывает данные, тем больше L, тем меньше (после минуса и логарифма) первое слагаемое;
- $s \cdot \log n$ — наказание за сложность модели. s — сколько чисел нужно задать, чтобы полностью описать модель, $\log n$ — множитель, растущий с объёмом данных. Смысл: сложную модель, которая идеально описывает обучающую выборку, но выучила шум, надо штрафовать. Иначе BIC всегда выбирал бы самую навороченную модель, и это вело бы к переобучению.

Добавление компонента выгодно, только если он реально улучшает описание данных **сильнее, чем дорожает модель**. Поэтому предпочтительна модель с наименьшим значением BIC:

$$K^* = \arg\min_K BIC(K).$$

Для GMM с полными ковариационными матрицами количество параметров модели определяется следующим выражением:

$$s = (K - 1) + K \cdot d + K \cdot \frac{d \cdot (d + 1)}{2},$$

где:

- $(K-1)$ — число независимых весов компонентов;
- $K \cdot d$ — число параметров векторов средних;
- $K\frac{d(d+1)}{2}$ — число параметров симметричных ковариационных матриц;
- K — количество компонентов;
- d — количество признаков.

Разберём каждое слагаемое в этом выражении.

1. Веса компонентов: $K-1$.

У каждого компонента есть вес π_k — доля точек, которые он «объясняет» (например, 60% трафика — `web`, 30% — `api`, 10% — `backup`). Всего весов K, но они должны в сумме давать 1, поэтому вес последнего компонента вычисляется из остальных весов: свободны только $K-1$ весов. Например, при $K=3$: π_1, π_2 свободны, а $\pi_3 = 1 - \pi_1 - \pi_2$

2. Центры облаков: $K \cdot d$.

Каждый компонент имеет свой центр — вектор средних μ_k — точку в пространстве признаков. Если признаков d, то вектор состоит из d чисел. Компонентов K, итого $K \cdot d$ чисел.

3. Формы облаков: $K \cdot \frac{d(d+1)}{2}$.

Каждый компонент имеет свою ковариационную матрицу Σ_k — она описывает форму и ориентацию облака: вытянутое, круглое, наклонное. Матрица имеет размер $d \times d$, но она симметричная: элемент (i, j) равен элементу (j, i). Поэтому независимых чисел в ней не d^2, а диагональ + один треугольник:

$$d + (d-1) + (d-2) + \cdots + 1 = \frac{d(d+1)}{2}.$$

Таких матриц K штук — по одной на компонент.

Шаг 2. Выбор формы ковариационных матриц.

В GMM ковариационная матрица Σ_k для k-го компонента — это параметр, который описывает разброс данных и взаимосвязи признаков внутри этого компонента. Она определяет форму, размер и ориентацию «облака» точек, соответствующего одной гауссовой компоненте.

Для каждого компонента необходимо определить способ представления ковариации признаков.

Полная ковариация.

Каждый компонент имеет собственную полную матрицу:

$$\Sigma_k = \begin{pmatrix} \mathrm{Var}(x_1) & \mathrm{Cov}(x_1, x_2) & \cdots & \mathrm{Cov}(x_1, x_d) \\ \mathrm{Cov}(x_2, x_1) & \mathrm{Var}(x_2) & \cdots & \mathrm{Cov}(x_2, x_d) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(x_d, x_1) & \mathrm{Cov}(x_d, x_2) & \cdots & \mathrm{Var}(x_d) \end{pmatrix},$$

где:

- $\mathrm{Var}(x_j)$ — дисперсия j-го признака внутри k-го компонента (насколько сильно значения этого признака отклоняются от среднего);
- $\mathrm{Cov}(x_i, x_j)$ — ковариация между i-м и j-м признаками (насколько совместно они отклоняются от своих средних).

Диагональная ковариация.

Матрица содержит только дисперсии:

$$\Sigma_k = \begin{pmatrix} \mathrm{Var}(x_1) & 0 & \dots & 0 \\ 0 & \mathrm{Var}(x_2) & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \mathrm{Var}(x_d) \end{pmatrix}.$$

Корреляции признаков не учитываются.

Сферическая ковариация.

«ферическая ковариация используется если облако точек имеет форму идеального шара: дисперсия по всем осям (признака) одинаковая, никаких наклонов и вытянутостей. И вместо целой матрицы чисел хранится одно-единственное число — дисперсия:

$$\Sigma_k = \mathrm{Var}_k \cdot I.$$

Здесь:

- Var_k — общая дисперсия компонента k;
- I — единичная матрица.

Ковариационная матрица описывает **форму облака точек** каждого компонента. Но реальные данные бывают разными: где-то режимы — плотные круглые пятна, где-то — вытянутые полосы, а где-то точек мало.

Как могут выглядеть облака в двумерных данных (признаки: длительность сессии и объём трафика):

- **Шар** — точки разбросаны одинаково во все стороны;
- **Эллипс вдоль осей** — вытянут, но не наклонён: объём растёт независимо от длительности;
- **Наклонный эллипс** — вытянут по диагонали: «длинная сессия = большой объём», признаки коррелируют.

Вид ковариации	Что задаёт	Форма облака	Параметров модели
Сферическая	одна дисперсия на компонент	шар, без корреляций	K
Диагональная	своя дисперсия **на каждую ось** внутри компонента	эллипс, оси параллельны осям признаков	$K \cdot d$
Полная	своя полная матрица у каждого компонента	любой эллипс у каждого, наклоны разные	$K \cdot \frac{d \cdot (d+1)}{2}$

Тип ковариационной матрицы — это гиперапаметр модели GMM и его как и количество компонент нужно подбирать экспериментальным путём.

Шаг 3. Инициализация параметров модели.

До начала обучения необходимо задать начальные значения:

- $\pi_k^{(0)}$ — начальный вес компонента;
- $\mu_k^{(0)}$ — начальный вектор средних;
- $\Sigma_k^{(0)}$ — начальная ковариационная матрица.

Начальные веса можно задать одинаковыми: $\pi_k^{(0)} = \frac{1}{K}$.

Начальные средние наиболее часто выбираются с помощью метода k-means.

Для выбора начальной ковариационной матрицы используются несколько способов. Если начальные средние определяются методом k-means, то логично взять и разброс из тех же кластеров:

$$\Sigma_k^{(0)} = \frac{1}{n_k} \sum_{x_i \in C_k} (x_i - \mu_k^{(0)})(x_i - \mu_k^{(0)})^\top,$$

- C_k — точки, которые k-means отнёс к кластеру k;
- n_k — их количество.

После инициализации запускается EM-алгоритм — Expectation Maximization.

Шаг 4. Расчёт плотности каждого объекта обучающей выборки для каждого компонента.

Для каждого объекта обучающей выборки x_i (наблюдение в наборе данных) и компонента k рассчитывается его многомерная нормальная плотность:

$$\mathcal{N}\left(x_i \mid \mu_k, \Sigma_k\right) = \frac{\exp\left[-\frac{1}{2}(x_i - \mu_k)^\top \Sigma_k^{-1}(x_i - \mu_k)\right]}{(2\pi)^{d/2}|\Sigma_k|^{1/2}}.$$

Выражение:

$$D_{ik}^2 = (x_i - \mu_k)^T \Sigma_k^{-1} (x_i - \mu_k)$$

является квадратом расстояния Махаланобиса между объектом x_i и центром k-го компонента.

Шаг 5. Е-шаг: вычисление ответственностей компонентов.

На Е-шаге для каждого объекта обучающей выборки x_i определяется вероятность его принадлежности каждому компоненту.

Ответственность компонента:

$$\gamma_{ik} = \frac{\pi_k \cdot \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{\ell=1}^{K} \pi_\ell \cdot \mathcal{N}(x_i \mid \mu_\ell, \Sigma_\ell)}.$$

По физическому смыслу ответственности - это апостериорные вероятности, которые показывают, с какой долей уверенности можно отнести объект x_i к k-му компоненту.

Ответственности удовлетворяют условиям: $0 \leq \gamma_{ik} \leq 1$ и $\sum_{k=1}^{K} \gamma_{ik} = 1$.

Например: $\gamma_{i1} = 0{,}80,\ \gamma_{i2} = 0{,}15,\ \gamma_{i3} = 0{,}05$ означает, что первый компонент даёт наибольшую вероятность наблюдать именно такой объект x_i при условии, что данные порождены этим гауссовым распределением.

Важное замечание. Большое значение γ_{ik} не доказывает, что объект нормальный.

Даже аномальный объект может иметь ответственность одного компонента, близкую к 1, если остальные компоненты объясняют его ещё хуже. Для обнаружения аномалии используется абсолютная плотность смеси, а не только ответственности.

Шаг 6. М-шаг: вычисление эффективного размера компонента.

Для каждого компонента рассчитывается эффективное число объектов:

$$N_k = \sum_{i=1}^{n} \gamma_{ik}.$$

Это не количество жёстко отнесённых объектов к компоненту, а суммарная «доля участия» всех объектов в этом компоненте.

N_k не обязательно является целым числом, поскольку объекты принадлежат компонентам вероятностно.

Шаг 7. М-шаг: обновление параметров компонента.

На этом шаге обновляются парметры каждого компонента:

- $\pi_k^{(new)}$ — новый вес компонента;
- $\mu_k^{(new)}$ — вектор вектор средних;
- $\Sigma_k^{(new)}$ — новая ковариационная матрица.

Вличина N_k является ключевой, поскольку именно через неё обновляются параметры компонента:

Новый вес компонента:

$$\pi_k^{\text{new}} = \frac{N_k}{n}.$$

Новый вектор средних:

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \cdot x_i.$$

Новая ковариационная матрица (для полной ковариации):

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} \cdot (x_i - \mu_k^{\text{new}}) \cdot (x_i - \mu_k^{\text{new}})^T.$$

Для численной устойчивости добавляется регуляризация:

$$\Sigma_k^{\text{regularized}} = \Sigma_k^{\text{new}} + \varepsilon I,$$

где:

- $\varepsilon > 0$ — небольшая положительная константа;
- I — единичная матрица размером $d \times d$.

Интерпретация в задачах кибербезопасности.

Допустим, есть $n = 10000$ сетевых сессий, и для компонента «сканирование портов» получилось $N_k = 320,5$.

Что это значит?

- Модель считает, что в данных примерно 320 сессий «по сути» относятся к режиму сканирования.
- Это даёт оценку частоты режима: $\pi_k^{\text{new}} = \frac{N_k}{n} = 0,032$, то есть около 3,2 % трафика соответствует этому профилю.
- Если в разные дни N_k сильно меняется - это может быть признаком аномальной активности (например, всплеск сканирований).

Шаг 8. Расчёт логарифма правдоподобия.

Логарифм правдоподобия всей выборки рассчитывается по следующему выражению:

$$\log L = \sum_{i=1}^{n} \log \left[\sum_{k=1}^{K} \pi_k \cdot \mathcal{N}\left(x_i \mid \mu_k, \Sigma_k\right) \right].$$

Чем больше $\log L$, тем выше суммарная плотность точек: данные хорошо укладываются в смесь гауссиан.

Чем меньше $\log L$, тем хуже модель описывает данные: типичные объекты имеют низкую плотность, то есть попадают в «редкие» области распределения.

В EM-алгоритме на каждой итерации обновляют параметры компонент так, чтобы $\log L$ возрастал. Это гарантирует, что модель становится всё лучше и лучше (по крайней мере локально).

Шаг 9. Проверка сходимости алгоритма.

Пусть логарифмы правдоподобия на двух последовательных итерациях равны: $\log L^{(t)}$ и $\log L^{(t-1)}$.

Обучение завершается, если: $\left|\log L^{(t)} - \log L^{(t-1)}\right| < \delta$.

Обучение также прекращается при достижении максимального числа итераций.

После проверки:

- если критерий выполнен — обучение завершается;
- если нет — алгоритм возвращается к E-шагу.

Шаг 10. Получение обученной модели.

После сходимости модель содержит для каждого компонента:

$$\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}.$$

Здесь:

- π_k — итоговый вес компонента;
- μ_k — итоговый вектор средних;
- Σ_k — итоговая ковариационная матрица;
- K — количество компонентов.

Эти параметры полностью определяют оценённую плотность:

$$\hat{p}(x) = \sum_{k=1}^{K} \hat{\pi}_k \cdot \mathcal{N}\left(x \mid \hat{\mu}_k, \hat{\Sigma}_k\right).$$

Символ $\hat{}$ обозначает параметры, оценённые по обучающим данным.

Шаг 11. Расчёт плотности нового события.

Пусть поступило новое сетевое событие: x_{new}.

Плотность нового объекта:

$$\hat{p}(x_{\text{new}}) = \sum_{k=1}^{K} \hat{\pi}_k \cdot \mathcal{N}\left(x_{\text{new}} \mid \hat{\mu}_k, \hat{\Sigma}_k\right).$$

Интерпретация:

- высокая плотность — событие соответствует одному или нескольким известным режимам;
- низкая плотность — событие плохо соответствует всем компонентам.

Шаг 12. Расчёт оценки аномальности.

Оценка аномальности определяется как отрицательный логарифм плотности:

$$S_{\mathrm{GMM}}\left(x_{\mathrm{new}}\right) = -\log \hat{p}\left(x_{\mathrm{new}}\right).$$

Интерпретация:

- маленький S_{GMM} — объект имеет высокую плотность;
- большой S_{GMM} — объект имеет низкую плотность (аномалия).

Важно понимать: $S_{\mathrm{GMM}}(x) \neq P(\text{атака} \mid x)$.

Это оценка статистической редкости, а не вероятность атаки.

Шаг 13. Выбор порога приянтяи решения об аномальности объекта.

Обучение без учителя.

Пусть ожидаемая доля аномалий равна c.

Порог определяется как:

$$T = Q_{1-c}\left(S_{\mathrm{GMM}}(X)\right).$$

Здесь:

- Q_{1-c} — квантиль уровня $1-c$;
- $S_{\mathrm{GMM}}(X)$ — оценки объектов смешанной выборки.

Нужно учитывать, что в режиме без учителя атаки участвуют в обучении GMM и могут стать отдельными компонентами.

Обучение только на нормальных данных.

Если модель обучена только на нормальной выборке, порог выбирается по независимым нормальным калибровочным объектам:

$$T = Q_{1-\alpha}\left(S_{\mathrm{GMM}}\left(X_{\mathrm{calibration}}\right)\right).$$

Шаг 14. Формирование итоговой метки.

Новое событие признаётся аномальным, если его оценка превышает порог:

$$\hat{y}(x) = \begin{cases} 1, & S_{\mathrm{GMM}}(x) > T; \\ 0, & S_{\mathrm{GMM}}(x) \leq T. \end{cases}.$$

Здесь:

- $\hat{y}(x)$ — предсказанная метка;

- $S_{\text{GMM}}(x)$ — оценка аномальности;
- T — порог;
- 1 — аномалия;
- 0 — нормальное событие.

В библиотеке `sklearn` метод GMM реализован классом `sklearn.mixture.GaussianMixture`.

Основные параметры:

- `n_components` (по умолчанию 1): число гауссовых компонент K;
- `covariance_type` (по умолчанию `full`): тип ковариационной матрицы Σ_k:
  - `full`: полная ковариационная матрица;
  - `tied`: полная ковариационная матрица с одинаковой формой разброса для всех компонентов;
  - `diag`: диагональная ковариационная матрица
  - `spherical`: сферическая ковариационная матрица;
- `tol` (по умолчанию 10^{-3}): порог сходимости EM (по приросту лог-правдоподобия);
- `n_init` (по умолчанию 1): сколько раз запустить EM с разными инициализациями и выбрать лучший $\log L$.

Основные методы:

- `fit(X)`: обучает модель с помощью EM-алгоритма:
  - считает веса π_k;
  - считает центры μ_k;
  - ситает мтарицу ковариаций Σ_k;
  - максимизирует лог-правдоподобие;
- `score_samples(X)`: возвращает логарифм плотности правдоподобия $\log L(x_i)$ для каждой точки. Это ключевой метод для детекции аномалий: низкие значения соответствуют редким точкам. Можно сразу использовать как score аномальности;
- `predict_proba(X)`: возвращает матрицу апостериорных вероятностей γ_{ik};
- `predict(X)`: возвращает метки компонентов: для каждой точки возвращает номер компонента с максимальной апостериорной вероятностью. В задачах обнаружения аномалий почти не используется напрямую.

Функция визуализации результатов детекции аномалий.

In [ ]:
```python
def plot_gmm_scores(data, score_column, threshold, title):
    """
    Строит распределения оценок нормальных событий и атак.

    Цвета соответствуют истинным меткам только для учебного
    анализа качества. При эксплуатации истинные метки неизвестны.
    """

    normal_scores = data.loc[data["is_attack"] == 0, score_column]
    attack_scores = data.loc[data["is_attack"] == 1, score_column]
```

```
    fig, ax = plt.subplots(figsize = (7, 5))
    ax.hist(normal_scores, bins = 30, alpha = 0.65,
            color = "steelblue", label="Нормальные события")

    ax.hist(attack_scores, bins = 20, alpha = 0.75,
            color = "red", label = "Атаки")

    ax.axvline(threshold, color = "black", linestyle = "--",
               linewidth = 2, label = f"Порог = {threshold:.2f}")

    ax.set_title(title)
    ax.set_xlabel(r"Оценка аномальности $-\log p(x)$")
    ax.set_ylabel("Количество событий")
    ax.grid(alpha=0.25)
    ax.legend()
    plt.tight_layout()
```

Реализация при обучении без учителя

При обучении без учителя GMM получает неразмеченный набор мониторинговых событий.

Модель не знает, какие объекты являются атаками. Она пытается описать всё наблюдаемое распределение смесью гауссовских компонентов.

Возникает важная проблема: если атаки образуют достаточно многочисленный и компактный кластер, GMM может выделить для них отдельную компоненту. Тогда внутри этой компоненты атакующие события будут иметь высокую плотность и не будут считаться аномалиями.

Будем использовать созданные при изучении метода KDE соответствующие выборки:

- `X_unsupervised` : стандартизованные мониторинговые данные (нормальные данные + атаки).

Обучение GMM.

In [ ]:
```
from sklearn.mixture import GaussianMixture

# Количество компонентов соответствует трём основным
# нормальным режимам нашего учебного датасета:
# web, api и backup.
N_COMPONENTS = 3

gmm_unsupervised = GaussianMixture(n_components = N_COMPONENTS, random_state = 4

    # Каждая компонента получает полную ковариационную матрицу.
    # Это позволяет учитывать зависимости между признаками.
    covariance_type = "full",

    # Небольшое значение добавляется к диагонали
    # ковариационной матрицы для численной устойчивости.
    reg_covar = 1e-6,
```

```
    # Используется одна инициализация EM.
    n_init = 1)

# В fit() передаются все мониторинговые события.
# Метки is_attack модели не передаются.
gmm_unsupervised.fit(X_unsupervised)

print("Алгоритм сошёлся:", gmm_unsupervised.converged_)
print("Количество итераций EM:", gmm_unsupervised.n_iter_)
```

```
Алгоритм сошёлся: True
Количество итераций EM: 4
```

Получение оценок аномальности.

То есть для каждого события вычисляется:

$$S_{\text{GMM}}(x) = -\log \hat{p}(x).$$

In [ ]:
```
# score_samples() возвращает log p(x):
# чем больше значение, тем событие более типично для модели.
log_density_unsupervised = gmm_unsupervised.score_samples(X_unsupervised)

# Меняем знак, чтобы получить стандартную интерпретацию:
# чем больше score, тем событие аномальнее.
gmm_unsupervised_scores = -log_density_unsupervised
gmm_unsupervised_scores[:5]
```

Out[ ]:
```
array([0.05385666, 1.59927211, 0.00705488, 2.37859534, 0.03168151])
```

Выбор порога.

Класс `GaussianMixture` не создаёт порог обнаружения аномалий автоматически.

В режиме без учителя можно задать предполагаемую долю аномалий. Предположим аналитик считает, что примерно 5,5% событий являются подозрительными.

In [ ]:
```
# Предполагаемая доля аномалий.
# В реальной системе это экспертная или эксплуатационная настройка.
expected_anomaly_fraction = 0.055

# Порог выбирается так, чтобы примерно 5,5% событий
# получили оценку выше порога.
gmm_unsupervised_threshold = np.quantile(gmm_unsupervised_scores,
                                         1 - expected_anomaly_fraction)

print(f'Порог GMM для режима без учителя: {gmm_unsupervised_threshold}')
```

```
Порог GMM для режима без учителя: 3.088300478987404
```

Недостаток такого подхода: это не порог допустимой частоты ложных тревог. Он лишь задаёт количество событий, которое система должна объявить аномальным.

Получение предсказаний.

In [ ]:
```
# Событие считается аномалией, если его оценка
# строго превышает выбранный порог.
gmm_unsupervised_prediction = (gmm_unsupervised_scores > gmm_unsupervised_thresh

# Сохраняем результаты в таблице.
monitoring_df["gmm_unsupervised_score"] = (gmm_unsupervised_scores)
monitoring_df["gmm_unsupervised_anomaly"] = (gmm_unsupervised_prediction)
```

In [ ]:
```
monitoring_df.head()
```

Out[ ]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	s
600	S0600	56.50	15.11	22.97	web	
601	S0601	57.21	19.07	26.54	web	
602	S0602	51.52	13.02	22.68	web	
603	S0603	40.72	14.97	30.16	web	
604	S0604	53.06	13.11	23.70	web	

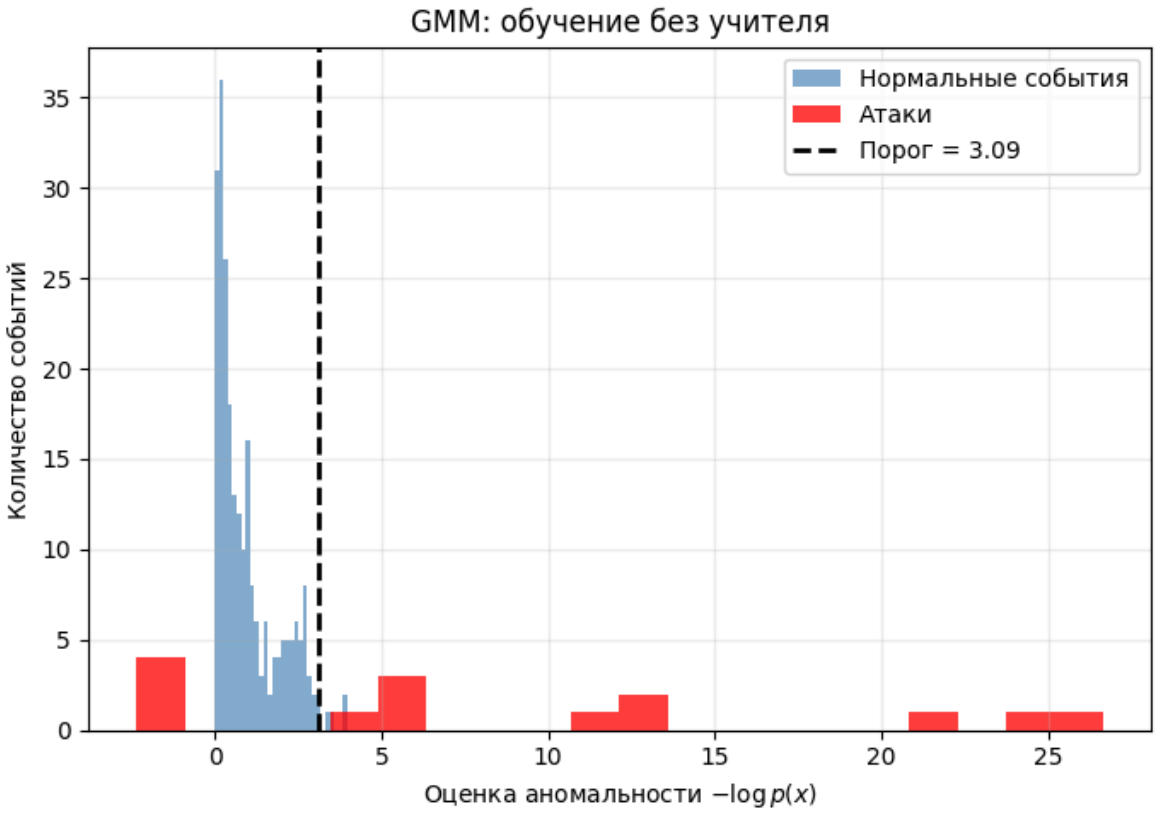
Оценивание качества детекции аномалий.

In [ ]:
```
print_detection_result(method_name = "GMM: обучение без учителя",
                       y_true = monitoring_df["is_attack"],
                       y_pred = monitoring_df["gmm_unsupervised_anomaly"])
```

```
GMM: обучение без учителя
TP — обнаруженные атаки: 10
FP — ложные тревоги: 4
FN — пропущенные атаки: 4
TN — корректно распознанная норма: 236
```

Визуализация результатов детекции аномалий.

In [ ]:
```
plot_gmm_scores(data = monitoring_df,
                score_column = "gmm_unsupervised_score",
                threshold = gmm_unsupervised_threshold,
                title = "GMM: обучение без учителя")
```



Реализация при обучении только на нормальных данных

В этом режиме модель строит описание только нормального поведения. Мониторинговые события, включая атаки, не участвуют.

Используем полученные ранее части нормальной выборки:

- `normal_fit` — обучение масштабирования и GMM;
- `normal_calibration` — выбор порога.

После масштабирования были получены выборки:

- `X_normal_fit` : стандартизованные значения обучающей выборки;
- `X_normal_calibration` : стандартизованные значения колибровочной выборки;
- `X_monitoring_scaled` : стандартизованные значения мониторнговой выборки.

Обучение GMM.

In [ ]:
```
gmm_normal = GaussianMixture(n_components = 3,
                             covariance_type = "full",
                             reg_covar = 1e-6,
                             random_state = 42,
                             n_init = 1)

# Модель получает исключительно нормальные события.
gmm_normal.fit(X_normal_fit)
```

```
print(f'Алгоритм сошёлся: {gmm_normal.converged_}')
print(f'Количество итераций EM: {gmm_normal.n_iter_}')
```

```
Алгоритм сошёлся: True
Количество итераций EM: 2
```

Оценки калибровочных данных.

Поскольку калибровочные события не участвовали в обучении, распределение `calibration_scores` показывает, какие оценки может получать новый нормальный трафик.

In [ ]:
```
# Оценки аномальности новых нормальных событий.
calibration_scores = -gmm_normal.score_samples(X_normal_calibration)
calibration_scores[:5]
```

Out[ ]:
```
array([ 0.17486103, -1.02176497,  3.86016207, -1.21606073,  1.86183828])
```

Выбор порога аномальности.

Порог имеет следующую интерпретацию:

Примерно 99% калибровочных нормальных событий имеют оценку аномальности не выше этого значения.

При условии репрезентативной калибровочной выборки ожидаемая частота ложных срабатываний составляет около: $1 - 0,99 = 0,01 = 1$

Это не гарантирует ровно 1% ложных тревог в будущем: распределение рабочего трафика может измениться.

In [ ]:
```
# Выберем 99-й процентиль.
normal_quantile = 0.99

gmm_normal_threshold = np.quantile(calibration_scores,
                                   normal_quantile)

print(f'Порог GMM при обучении только на норме: {gmm_normal_threshold}')
```

```
Порог GMM при обучении только на норме: 4.87022082532893
```

Анализ мониторинговых событий.

In [ ]:
```
# Вычисляем оценки для мониторинговых событий.
gmm_normal_scores = -gmm_normal.score_samples(X_monitoring)

# Сравниваем их с порогом, полученным
# исключительно по отложенным нормальным данным.
gmm_normal_prediction = (gmm_normal_scores > gmm_normal_threshold).astype(int)

# Сохраняем результаты.
monitoring_df["gmm_normal_score"] = gmm_normal_scores
monitoring_df["gmm_normal_anomaly"] = gmm_normal_prediction
monitoring_df.head()
```

Out[ ]:

	session_id	connections_per_min	unique_destinations	outbound_mb	traffic_type	s
600	S0600	56.50	15.11	22.97	web	
601	S0601	57.21	19.07	26.54	web	
602	S0602	51.52	13.02	22.68	web	
603	S0603	40.72	14.97	30.16	web	
604	S0604	53.06	13.11	23.70	web	

Оценивание качества детекции аномалий.

In [ ]:
```
print_detection_result(method_name = "GMM: обучение только на нормальных данных"
                       y_true = monitoring_df["is_attack"],
                       y_pred = monitoring_df["gmm_normal_anomaly"])
```

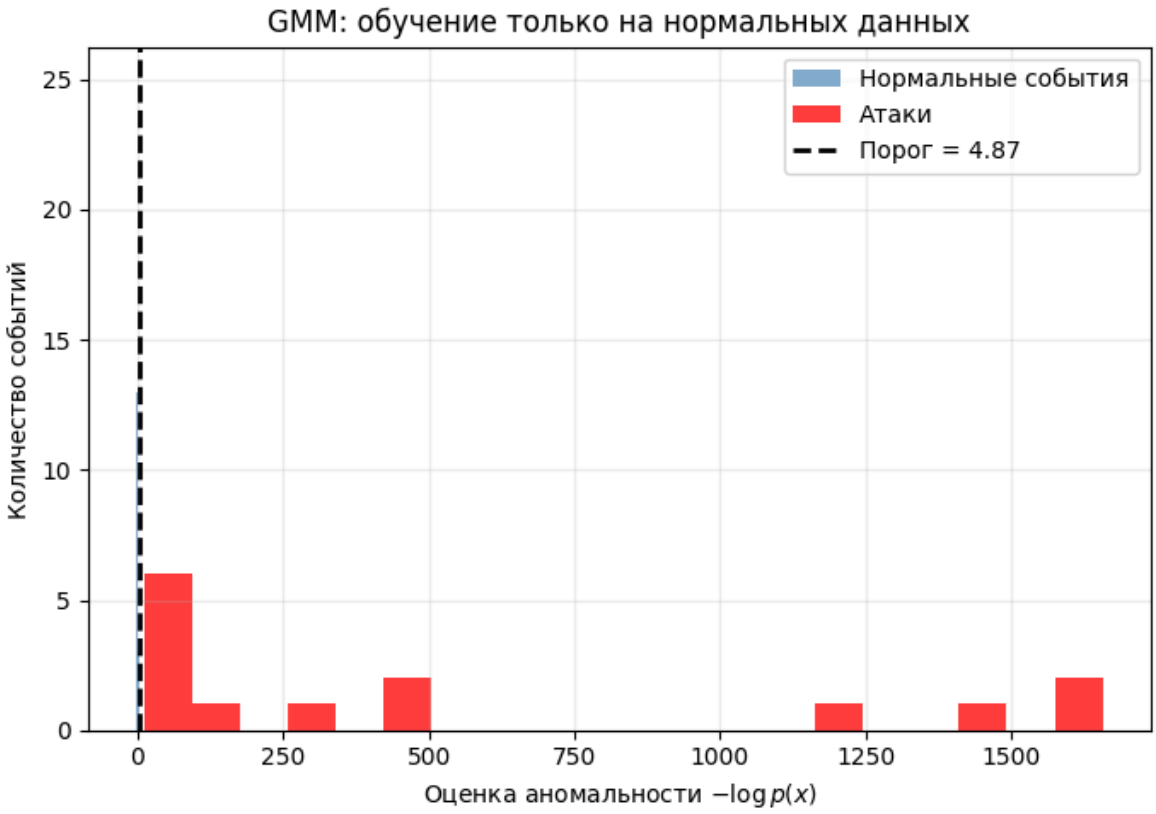
```
GMM: обучение только на нормальных данных
TP — обнаруженные атаки: 14
FP — ложные тревоги: 1
FN — пропущенные атаки: 0
TN — корректно распознанная норма: 239
```

В этом режиме обнаруживаются все четыре типа атак, включая сканирование портов.

Причина заключается в том, что GMM не видел сканирования во время обучения. Следовательно, модель не могла создать для него отдельную гауссовскую компоненту.

Визуализация результатов детекции аномалий.

In [ ]:
```
plot_gmm_scores(data = monitoring_df,
                score_column = "gmm_normal_score",
                threshold = gmm_normal_threshold,
                title = "GMM: обучение только на нормальных данных")
```



Ограничения метода GMM

1. Предположение о гауссовской форме компонентов.

Каждая компонента GMM описывается многомерным нормальным распределением. Поэтому модель лучше работает, когда отдельные режимы нормального поведения имеют приблизительно эллиптическую форму.

В реальном сетевом трафике распределения часто бывают:

- асимметричными;
- имеющими тяжёлые хвосты;
- ограниченными нулём;
- дискретными;
- содержащими несколько пиков;
- имеющими сложную нелинейную форму.

Например, распределение объёма исходящего трафика может быть сильно скошено вправо: большинство сеансов передаёт небольшой объём данных, но иногда выполняется легитимное резервное копирование.

В этом случае GMM может:

- плохо описать нормальные данные;
- назначить низкую плотность легитимным событиям;
- создать большое количество ложных тревог.

Возможные меры:

- логарифмирование сильно скошенных признаков;
- разделение разных типов трафика;
- увеличение количества компонентов;
- применение KDE или методов, не предполагающих определённую форму распределения.

2. Необходимость заранее выбирать количество компонентов.

Параметр `n_components` задаётся до обучения.

Слишком маленькое количество компонентов приводит к объединению различных нормальных режимов. Например, веб-трафик, API-запросы и резервное копирование могут быть описаны одной слишком широкой компонентой.

Последствия:

- модель становится нечувствительной к атакам;
- области с низкой фактической плотностью получают завышенную плотность;
- увеличивается количество пропущенных атак.

Слишком большое количество компонентов приводит к переобучению:

- компоненты начинают описывать случайные колебания;
- отдельная компонента может сформироваться вокруг группы атак;
- модель становится нестабильной;
- возрастают требования к объёму данных.

Для выбора числа компонентов используются:

- BIC;
- AIC;
- знания о режимах работы системы;
- оценка качества на проверочной выборке.

3. GMM может моделировать атаки как отдельный нормальный компонент.

Это особенно важно в режиме обучения без учителя.

Если атаки присутствуют в обучающей выборке и образуют компактную группу, GMM может выделить для них отдельную компоненту. После этого атакующие объекты получают высокую плотность внутри своей компоненты и перестают считаться аномалиями.

Это называется маскированием аномалий: аномальные объекты поддерживают плотность друг друга.

Возможные меры:

- обучать GMM только на нормальных данных;
- очищать обучающую выборку;
- ограничивать количество компонентов;

- периодически проверять смысл обнаруженных компонентов;
- использовать размеченные атаки для контрольного тестирования.

4. Чувствительность к загрязнению эталонной нормальной выборки.

Обучение только на нормальных данных уменьшает риск моделирования атак, но предполагает, что эталонная выборка действительно содержит только нормальные события.

Если в неё попали незамеченные атаки, то параметры смеси изменяются:

- средние значения компонентов смещаются;
- ковариации увеличиваются;
- области атак могут получить высокую плотность;
- порог становится завышенным.

Следовательно, перед обучением необходимо:

- проверять эталонные данные;
- исключать периоды известных инцидентов;
- использовать данные из нескольких проверенных периодов;
- контролировать состав компонентов после обучения.

5. В GMM отсутствует встроенный порог аномальности.

`GaussianMixture` является моделью плотности, а не готовым классификатором аномалий. Он не определяет, с какого значения объект нужно считать атакой. Порог приходится выбирать отдельно.

Порог зависит от:

- допустимой частоты ложных тревог;
- состава калибровочных данных;
- режима работы инфраструктуры;
- стоимости пропуска атаки;
- изменения сетевого трафика со временем.

Слишком низкий порог создаёт много ложных тревог. Слишком высокий — приводит к пропуску атак.

6. Чувствительность к изменению нормального поведения.

Сетевой трафик не является стационарным. Его распределение меняется из-за:

- появления новых сервисов;
- изменения числа пользователей;
- обновления программного обеспечения;
- перехода сотрудников на удалённую работу;
- изменения расписания резервного копирования;
- сезонных нагрузок.

Это явление называется дрейфом данных или концептуальным дрейфом.

Модель, обученная на старых данных, может:

- считать новый легитимный режим атакой;
- перестать обнаруживать атаки, похожие на изменившуюся норму;
- генерировать растущее количество ложных тревог.

Поэтому необходимо:

- контролировать распределения признаков;
- анализировать динамику оценок GMM;
- периодически перекалибровывать порог;
- переобучать модель на проверенных нормальных данных;
- сохранять старые модели для сравнения.

Автоматическое переобучение на всех новых событиях опасно: модель может постепенно включить продолжающуюся атаку в нормальное поведение.

5. Histogram-Based Outlier Score (HBOS)

Метод строит отдельную гистограмму для каждого признака.

Суть метода заключается в том, чтобы для каждой точки данных оценить, насколько редко встречаются её значения по каждому признаку в отдельности, а потом суммирует эти «редкости» по всем признакам. Если по нескольким признакам сразу значения попадают в редкие (малочисленные) бины гистограммы — точка получает высокий балл аномальности.

Алгоритм метода

Пусть есть набор данных из n объектов: $X = x_1, \dots, x_i, \dots x_n$. Каждый объект описывается d признаками. Для каждого объекта $x_i = (x_{i1}, x_{i2}, \dots, x_{ij}, \dots, x_{id})$, где x_{ij} — значение j-го признака у i-го объекта.

Цель HBOS — для каждой точки x_i посчитать скалярный балл аномальности $\mathrm{HBOS}(x_i)$: чем он выше, тем более точка «редкая» по совокупности своих значений в одномерных гистограммах.

Ключевое предположение: признаки условно независимы:

$$f(x_{i1}, x_{i2}, \dots, x_{ij}, \dots, x_{id}\ ) \approx \prod_{i=j}^{d} f(x_{ij}),$$

т. е. совместная функция плотности распределения признаков равна произведению функций плотности отдельных признаков.

Это позволяет строить отдельные гистограммы по каждому признаку и затем объединять их.

Оценкой одномерной плотности является частота попадания значений признака в каждый интервал.

Шаг 1. Построение одномерных гистограмм по каждому признаку.

Для каждого признака $j = 1, \ldots, d$ строится гистограмма:

- диапазон значений этого признака разбивается на m_j бинов (интервалов);
- для каждого бина вычисляется количество попавших в него значений признака.

Чаще всего используют бины одинаковой ширины:

$$\text{ширина бина } h_j = \frac{\max(x_{ij}) - \min(x_{ij})}{m_j}.$$

Шаг 2. Оценка локальной плотности (вероятности) для каждого бина.

Для признака j и бина номер b эмпирическая вероятность попадания в этот бин:

$$p_{jb} = \frac{n_{jb}}{n},$$

где:

- n_{jb} - количество значений признака j, которые попали в бин b;
- n - объем выборки.

Чтобы избежать деления на ноль (если есть пустые бины) добавлется сглаживание Лапласа:

$$p_{jb} = \frac{n_{jb} + \alpha}{n + m_j},$$

где α — параметр сглаживания (например, $\alpha = 0.1$ или 1). В базовом варианте можно считать $\alpha = 0$.

Шаг 3. Нормализация гистограмм (для сопоставимости признаков).

Поскольку разные признаки могут иметь разную «разреженность» и разное число бинов, для устойчивости полезно нормировать оценки вероятностей внутри каждого признака так, чтобы максимальная вероятность по бинам была равна 1:

$$\tilde{p}_{jb} = \frac{p_{jb}}{\max(p_{jb})}.$$

Тогда $\tilde{p}_{jb} \in (0, 1]$, и редкие бины имеют малые значения.

Шаг 4. Вычисление «сырого» балла аномальности для объекта.

Для объекта x_i определяется, в какой бин b_{ij} попал его j-й признак:

$$b_{ij} = \left\lfloor \frac{x_{ij} - \min(x_{ij})}{h_j} \right\rfloor + 1,$$

где:

- x_{ij} — проверяемое значение;
- $\min(x_{ij})$ — минимальное значение признака j;
- h_j — ширина интервала для признака j;
- $\lfloor \cdot \rfloor$ — округление вниз до целого.

Теперь для каждого признака берется нормированная вероятность этого бина $\tilde{p}_{jb_{ij}}$ - чем она меньше, тем более аномальным считается значение признака.

Для получения совместной вероятности появлений значений признака для i-го объекта на пратике используется способ агрегирования, основанный на логарифмировании произведений совместной функции плотности:

$$\mathrm{HBOS}(x_i) = \sum_{j=1}^{d} -\log(\tilde{p}_{jb_{ij}}).$$

Шаг 5. Интерпретация и выбор порога.

Балл $\mathrm{HBOS}(x_i)$ — это суммарная «редкость» точки по всем признакам. Чем выше балл, тем более аномальна точка.

Порог выбирают одним из способов:

- **Квантиль**: например, точки с HBOS выше 99-го перцентиля по выборке считаются аномалиями.
- **Фиксированный порог**: если известно, что при типичных значениях балл не превышает некоторого значения.
- **Динамический порог**: по скользящему окну (для временных рядов/потоков).

Функции для реализации метода HBOS

In [ ]:
```python
# Входные данные:
# - X: двумерный массив (строки — объекты, столбцы — признаки);
# - bins: количество бинов;
# - alpha: параметр сглаживания

def fit_hbos(X, bins = 12, alpha = 1.0):
    models = []      # Список, в который будем сохранять параметры гистограмм дл
    n_objects = len(X)  # Общее количество объектов (строк) в выборке X

    # Проходим по каждому признаку (столбцу) матрицы X
    for feature_index in range(X.shape[1]):
        # Строим гистограмму для текущего признака:
        # counts — количество объектов в каждом бине (массив длины bins)
        # edges — границы бинов (массив длины bins+1)
        counts, edges = np.histogram(X[:, feature_index], bins=bins)
```

```
        # Вычисляем оценки вероятностей попадания в каждый бин с применением сгл
        probabilities = (counts + alpha) / (n_objects + alpha * bins)

        # Сохраняем границы бинов и вероятности для текущего признака в виде сло
        models.append({"edges": edges, # Границы бинов (нужны для отнесения новы
                       "probabilities": probabilities})  # Вероятности попадания

    # Результат: список моделей (по одной на признак) с границами бинов и сглаже
    return models

# Входные данные:
# - models: список обученных моделей (гистограмм) по каждому признаку, полученны
# - X: новые данные (или те же данные), для которых нужно посчитать скоры аномал
# - epsilon: малое число для защиты от логарифма нуля (если вероятность вдруг ок

def hbos_score_samples(models, X, epsilon = 1e-12):
    # Создаём массив скоров длиной, равной количеству объектов в X (изначально в
    scores = np.zeros(len(X))

    # Проходим по всем признакам и соответствующим обученным моделям
    for feature_index, model in enumerate(models):
        edges = model["edges"]                  # Границы бинов, полученные на э
        probabilities = model["probabilities"]  # Сглаженные вероятности для бин

        # Для каждого объекта находим индекс бина, в который попадает значение п
        # np.searchsorted ищет позицию вставки значения в отсортированный массив
        # side="right" означает, что при совпадении значение относится к правому
        # Вычитаем 1, потому что searchsorted возвращает индекс справа, а нам ну
        bin_indices = np.searchsorted(edges, X[:, feature_index], side = "right"

        # Особый случай: если значение точно равно правой границе последнего бин
        # то searchsorted вернёт индекс за пределами массива, а после -1 получит
        # Но для надёжности явно относим такие точки к последнему бину.
        bin_indices[X[:, feature_index] == edges[-1]] = len(probabilities) - 1

        # Маска (булев массив), показывающая, какие индексы бинов корректны (в п
        inside_range = ((bin_indices >= 0) & (bin_indices < len(probabilities)))

        # Инициализируем массив вероятностей для всех объектов значением epsilon
        point_probabilities = np.full(len(X), epsilon)

        # Для объектов, попавших в допустимые бины, подставляем реальные вероятн
        point_probabilities[inside_range] = (probabilities[bin_indices[inside_ra

        # Считаем вклад текущего признака в общий скор аномальности:
        # HBOS использует сумму отрицательных логарифмов вероятностей: -log(p).
        # Чем меньше вероятность (редче значение), тем больше скор.
        scores -= np.log(point_probabilities)

    return scores  # Возвращаем итоговые скоры аномальности (чем больше значение
```

Реализация при обучении без учителя

В этом режиме HBOS ищет выбросы внутри мониторинговой выборки.

Будем использовать созданные при изучении метода KDE соответствующие выборки:

- `X_unsupervised` : стандартизованные мониторинговые данные (нормальные данные + атаки).

Обучим «ручную» реализацию HBOS на данных мониторинговой выборки (смешанные данные: норма и атаки без меток).

In [ ]:
```
# Формируем список моделей (по одной на признак) с границами бинов и сглаженными
hbos_unsupervised_model = fit_hbos(X_unsupervised, bins = 12, alpha = 1.0)

# Считаем скоры аномальности для тех же данных, используя обученные гистограммы
# Результат: массив чисел, где большее значение означает более высокую степень а
hbos_unsupervised_score = hbos_score_samples(hbos_unsupervised_model, X_unsuperv

# При известной доле аномалий (contamination) выбираем порог через квантиль скор
# Логика: если contamination=0.05 (5% аномалий), то порог — это 95-й процентиль
# Все точки со скором выше порога считаем аномалиями.
contamination = 0.05
hbos_unsupervised_threshold = np.quantile(hbos_unsupervised_score, 1 - contamina

# Формируем бинарные предсказания: True (аномалия) там, где скор превышает порог
# Результат: булев массив (True/False), который можно трактовать как метки 1/0
hbos_unsupervised_prediction = (hbos_unsupervised_score > hbos_unsupervised_thre

# Выводим рассчитанный порог, чтобы понимать, насколько строгим получился детект
print(f"Порог HBOS: {hbos_unsupervised_threshold:.3f}")
```

```
Порог HBOS: 6.086
```

Оценим качество работы метода.

In [ ]:
```
# monitoring_df["is_attack"] — истинные метки: 1 — атака, 0 — норма
# hbos_normal_prediction — предсказанные метки (True=аномалия, False=норма)
print_detection_result("HBOS: обучение без учителя",
                       monitoring_df["is_attack"],
                       hbos_unsupervised_prediction)
```

```
HBOS: обучение без учителя
TP — обнаруженные атаки: 8
FP — ложные тревоги: 1
FN — пропущенные атаки: 6
TN — корректно распознанная норма: 239
```

Какие атаки были не выявлены.

In [ ]:
```
monitoring_df['HBOS_unsupervised_anomaly'] = hbos_unsupervised_prediction
monitoring_df[['scenario', 'HBOS_unsupervised_anomaly']][monitoring_df['scenario
```

Out[ ]:

	scenario	HBOS_unsupervised_anomaly
840	port_scan	True
841	port_scan	True
842	port_scan	True
843	port_scan	True
844	exfiltration	False
845	exfiltration	True
846	exfiltration	False
847	beaconing	False
848	beaconing	False
849	beaconing	False
850	beaconing	False
851	lateral_movement	True
852	lateral_movement	True
853	lateral_movement	True

Модель пропустила все атаки типа `beaconing`.

`Beaconing` — это периодическое обращение заражённого устройства к управляющему серверу злоумышленника.

Обычно вредоносная программа:

- устанавливает соединение с одним внешним адресом;
- передаёт небольшой объём данных;
- ждёт некоторое время;
- повторяет соединение.

В нашем датасете события `beaconing` имеют приблизительные значения:

- `connections_per_min ≈ 70–81`;
- `unique_destinations ≈ 1`;
- `outbound_mb ≈ 3`.

Дел в том, что отдельные значения каждого признака не являются аномальными, но их сочетание является подозрительным: относительно много соединений с одним адресатом и очень маленьким объёмом передаваемых данных.

Как HBOS анализирует событие:

- строит отдельную гистограмму для каждого признака;
- затем он отдельно определяет редкость каждого значения.

Проблема заключается в предположении независимости признаков. Поэтому метод оценивает, насколько редко встречается каждое значение по отдельности, но плохо проверяет, характерно ли их совместное появление.

Анализ отдельных признаков `beaconing`.

Количество соединений `connections_per_min ≈ 70–81`. Такое число соединений не является экстремальным. Оно находится относительно близко к обычному веб-трафику: `connections_per_min ≈ 55`. Поэтому первый признак не получает очень большой аномальный балл.

Число уникальных адресатов `unique_destinations ≈ 1`. Малое число адресатов встречается при резервном копировании(`backup`): `unique_destinations ≈ 2`. Следовательно, и это значение само по себе не выглядит невозможным.

Объём исходящего трафика `outbound_mb ≈ 3`. Это небольшой объём данных. Небольшие значения могут встречаться среди обычных коротких сетевых сессий. Поэтому третий признак также не получает достаточно большого аномального балла.

Реализация при обучении только на нормальных данных

Будем использовать уже подготовленные наборы данных:

- `X_normal_fit` : стандартизованные значения обучающей выборки;
- `X_normal_calibration` : стандартизованные значения колибровочной выборки;
- `X_monitoring` : стандартизованные значения мониторинговой выборки.

Обучим «ручную» реализацию HBOS только на подтверждённо нормальных данных (X_normal_fit).

In [ ]:
```
# Формируем список моделей (по одной на признак) с границами бинов и сглаженными
hbos_normal_model = fit_hbos(X_normal_fit, bins = 12, alpha = 1.0)

# Считаем скоры аномальности на калибровочной нормальной выборке.
# Результат: массив скоров для нормальных точек; по их распределению будем выбир
hbos_calibration_score = hbos_score_samples(hbos_normal_model, X_normal_calibrat

# Считаем скоры на данных мониторинга (норма + возможные атаки).
# Эти скоры будут нужны для оценивания качества модели
# Используем тот же профиль нормы (hbos_normal_model), чтобы оценить, насколько
hbos_monitoring_score = hbos_score_samples(hbos_normal_model, X_monitoring)

# Подбираем порог по 99-му процентилю скоров на калибровочной НОРМАЛЬНОЙ выборке
# Смысл: допускаем не более 1% ложных срабатываний на нормальном трафике.
# Такой подход даёт контролируемый уровень ложных срабатываний — критично для SO
# Результат: пороговое значение; всё, что выше него, считаем аномалией
hbos_normal_threshold = np.quantile(hbos_calibration_score, 0.99)

# Формируем бинарные предсказания для мониторинговых данных.
```

```
# Если скор сессии выше порога — помечаем как аномалию (True), иначе — норма (Fa
hbos_normal_prediction = (hbos_monitoring_score > hbos_normal_threshold)

# Выводим рассчитанный порог.
print(f"Порог HBOS: {hbos_normal_threshold:.3f}")
```

```
Порог HBOS: 10.619
```

Оценим качество работы метода.

In [ ]:
```
# monitoring_df["is_attack"] — истинные метки: 1 — атака, 0 — норма
# hbos_normal_prediction — предсказанные метки (True=аномалия, False=норма)
print_detection_result("HBOS: обучение только на нормальных данных",
                       monitoring_df["is_attack"],
                       hbos_normal_prediction)
```

```
HBOS: обучение только на нормальных данных
TP — обнаруженные атаки: 10
FP — ложные тревоги: 3
FN — пропущенные атаки: 4
TN — корректно распознанная норма: 237
```

Какие атаки были не выявлены.

In [ ]:
```
monitoring_df['HBOS_normal'] = hbos_normal_prediction
monitoring_df[['scenario', 'HBOS_normal']][monitoring_df['scenario'] != 'normal'
```

Out[ ]:

	scenario	HBOS_normal
840	port_scan	True
841	port_scan	True
842	port_scan	True
843	port_scan	True
844	exfiltration	True
845	exfiltration	True
846	exfiltration	True
847	beaconing	False
848	beaconing	False
849	beaconing	False
850	beaconing	False
851	lateral_movement	True
852	lateral_movement	True
853	lateral_movement	True

HBOS обнаруживает:

- все сканирования;
- все случаи эксфильтрации;

- все случаи бокового перемещения.

Четыре события `beaconing` снова пропускаются.

Реализация с использованием библиотеки PyOD

Метод HBOS реализован в классе `HBOS` модуля `models` бибилиотеки `pyod`.

Основные параметры:

- `n_bins` (по умолчанию 10): число бинов в гистограмме для каждого признака;
- `alpha` (по умолчанию 0.1): параметр сглаживания, чтобы избежать нулевых вероятностей;
- `contamination` (по умолчанию 0.1): ожидаемая доля аномалий в данных; используется для автоматического перевода скоров в бинарные метки.

Основные методы:

- `fit(X)` — обучает модель на данных X. Для HBOS это означает построение гистограмм по каждому признаку;
- `decision_function(X)` — возвращает непрерывный скор аномальности (чем больше — тем подозрительнее);
- `predict(X)` — возвращает бинарные метки: 1 — аномалия, 0 — норма. Пороговое значение берётся из `contamination`;
- `predict_proba(X)` — возвращает оценку вероятности аномальности (в диапазоне [0,1]). Полезно, когда нужно отображать «уверенность» модели.

В обоих режимах применяется один класс `HBOS`.

Различаются:

- данные, передаваемые в fit();
- способ получения прогнозов;
- смысл параметра `contamination`;
- способ выбора порога.

Реализация при обучении без учителя

В учебном датасете 14 атак из 254 событий. Истинная доля атак: 14/254≈0,0551.

В учебном примере это значение можно использовать как `contamination`. В реальной задаче точная доля атак обычно неизвестна.

In [ ]:
```
# @title
!pip install pyod
```

In [ ]:
```
from pyod.models.hbos import HBOS
```

```
# В учебном эксперименте доля аномалий известна.
contamination = (monitoring_df["is_attack"].mean())

hbos_unsupervised = HBOS(n_bins = 12,
                         alpha = 0.1,
                         contamination = contamination)

# Модель обучается на смешанной выборке:
# нормальные события + неизвестные модели атаки.
hbos_unsupervised.fit(X_unsupervised)
```

Out[ ]:

HBOS

```
HBOS(alpha=0.1, contamination=np.float64(0.05511811023622047), n_bins=12,
   tol=0.5)
```

После обучения PyOD автоматически создаёт три атрибута.

Непрерывные оценки аномальности обучающих объектов (для всех тех же объектов, которые участвовали в обучении (построении гистограмм)) - `decision_scores_`.

In [ ]:
```
unsupervised_scores = (hbos_unsupervised.decision_scores_)
unsupervised_scores[:10]
```

Out[ ]:
```
array([ 0.26930701,  0.26930701, -1.00033926,  2.16380559, -1.00033926,
       -1.00033926,  0.26930701, -1.00033926, -1.00033926,  0.89415931])
```

Автоматически рассчитанный порог принятия решений - `threshold_`.

In [ ]:
```
unsupervised_threshold = (hbos_unsupervised.threshold_)
unsupervised_threshold
```

Out[ ]:
```
np.float64(3.739587038721726)
```

Метки аномальности для обучающей выборки - `labels_`: 0 — нормальный объект; 1 — аномалия.

In [ ]:
```
unsupervised_prediction = (hbos_unsupervised.labels_)
unsupervised_prediction[:20]
```

Out[ ]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Полный код обучения модели в режиме обучения без учителя.

In [ ]:
```
hbos_unsupervised = HBOS(n_bins=12, alpha=0.1, contamination = contamination)
hbos_unsupervised.fit(X_unsupervised)
unsupervised_scores = (hbos_unsupervised.decision_scores_)
unsupervised_threshold = (hbos_unsupervised.threshold_)
unsupervised_prediction = (hbos_unsupervised.labels_)

monitoring_df["hbos_pyod_unsupervised_score"] = unsupervised_scores
monitoring_df["hbos_pyod_unsupervised_anomaly"] = unsupervised_prediction
```

Оценим качество работы метода.

In [ ]:
```
print_detection_result("HBOS PyOD: обучение без учителя",
                       monitoring_df["is_attack"],
                       unsupervised_prediction)
```

```
HBOS PyOD: обучение без учителя
TP — обнаруженные атаки: 5
FP — ложные тревоги: 9
FN — пропущенные атаки: 9
TN — корректно распознанная норма: 231
```

Какие атаки были не выявлены.

In [ ]:
```
monitoring_df[['scenario',
               'hbos_pyod_unsupervised_score',
               'hbos_pyod_unsupervised_anomaly']][monitoring_df['scenario'] != '
```

Out[ ]:

	scenario	hbos_pyod_unsupervised_score	hbos_pyod_unsupervised_anomaly
840	port_scan	5.364521	1
841	port_scan	5.364521	1
842	port_scan	5.509503	1
843	port_scan	5.364521	1
844	exfiltration	2.711893	0
845	exfiltration	2.792054	0
846	exfiltration	2.711893	0
847	beaconing	0.988358	0
848	beaconing	0.988358	0
849	beaconing	0.988358	0
850	beaconing	0.988358	0
851	lateral_movement	3.720284	0
852	lateral_movement	3.720284	0
853	lateral_movement	4.766985	1

Реализация при обучении только на нормальных данных

Будем использовать уже подготовленные наборы данных:

- X_normal_fit: стандартизованные значения обучающей выборки;
- X_normal_calibration: стандартизованные значения колибровочной выборки;
- X_monitoring: стандартизованные значения мониторинговой выборки.

In [ ]:
```
#Обучение HBOS
hbos_normal = HBOS(n_bins = 12, alpha = 0.1,
                   # PyOD требует допустимое значение contamination.
                   # Его внутренний порог ниже использовать не будем.
                   contamination = 0.05)

hbos_normal.fit(X_normal_fit)

# Непрерывная оценка аномальности калибровочных данных.
# Калибровочные события не участвовали в построении гистограмм.
# Поэтому они имитируют новые нормальные сетевые сессии.
calibration_scores = (hbos_normal.decision_function(X_normal_calibration))

# Ручной выбор порога (например, выберем 97-й процентиль):
manual_threshold = np.quantile(calibration_scores, 0.97)

# Непрерывная оценка аномальности мониторинговых событий (для оценивания качеств
monitoring_scores = (hbos_normal.decision_function(X_monitoring))
# Предсказанные метки для мониторинговых событий (1 — атака, 0 — норма)
monitoring_prediction = (monitoring_scores > manual_threshold).astype(int)

# Запись результатов в таблицу
monitoring_df["hbos_pyod_normal_score"] = monitoring_scores
monitoring_df["hbos_pyod_normal_anomaly"] = monitoring_prediction
```

In [ ]:
```
print_detection_result("HBOS PyOD: обучение только на норме",
                       monitoring_df["is_attack"],
                       monitoring_prediction)
```

```
HBOS PyOD: обучение только на норме
TP — обнаруженные атаки: 4
FP — ложные тревоги: 2
FN — пропущенные атаки: 10
TN — корректно распознанная норма: 238
```

In [ ]:
```
print(f"Калибровочный порог: {manual_threshold:.3f}")

monitoring_df[['scenario',
               'hbos_pyod_normal_score',
               'hbos_pyod_normal_anomaly']][monitoring_df['scenario'] != 'normal
```

```
Калибровочный порог: 6.076
```

Out[ ]:

	scenario	hbos_pyod_normal_score	hbos_pyod_normal_anomaly
840	port_scan	6.350963	1
841	port_scan	6.350963	1
842	port_scan	6.350963	1
843	port_scan	6.350963	1
844	exfiltration	4.308523	0
845	exfiltration	4.247174	0
846	exfiltration	4.308523	0
847	beaconing	3.939069	0
848	beaconing	3.939069	0
849	beaconing	5.130878	0
850	beaconing	3.939069	0
851	lateral_movement	4.384984	0
852	lateral_movement	3.541817	0
853	lateral_movement	5.293276	0

Ограничения метода HBOS

1. Сильная зависимость от числа бинов (n_bins).

Качество детекции критически зависит от выбора количества бинов. При слишком малом числе бинов теряется детализация: аномальные точки «растворяются» в широких интервалах. При слишком большом — в бинах оказывается мало точек, распределение становится шумным, и нормальные точки могут получать высокие скоры аномальности.

2. Предпоположение о независимости признаков.

HBOS оценивает плотность по каждому признаку отдельно и суммирует логарифмы вероятностей. Это эквивалентно предположению, что признаки независимы. Это приводит к тому, что аномалии, которые проявляются только в совместном поведении признаков (например, «небольшое число соединений, но все — на редкие порты»), HBOS может не заметить. Такие паттерны хорошо ловит, например, расстояние Махаланобиса или LOF.

3. Чувствительность к выбросам в обучающей выборке.

Если в данных, на которых строится профиль нормы (X_normal_fit), случайно попадут реальные атаки, гистограммы «размажутся», и порог станет менее строгим. В итоге будущие атаки могут выглядеть «нормально».

Критично для SOC: модель должна обучаться только на подтверждённо легитимном трафике. Если такой выборки нет, лучше использовать полууправляемые или полностью неконтролируемые методы, но с осторожностью к порогам. Как смягчить.

4. Ограниченная интерпретируемость вклада взаимодействий.

Можно легко сказать, какой признак дал наибольший вклад в скор аномальности (потому что скор — сумма по признакам). Но нельзя объяснить аномалию через взаимодействие признаков: «аномалия потому, что X и Y одновременно такие-то». Для расследования инцидентов. Это снижает ценность HBOS в случаях, когда аналитик хочет понять именно совместное поведение признаков.

Литература

1. Чио К., Фримэн Д. Машинное обучение и безопасность [Электронный ресурс]:. - Москва: ДМК Пресс, 2020. - 388 с. – Режим доступа: https://e.lanbook.com/book/131707.
2. Протодьяконов А. В., Пылов П. А., Садовников В. Е. Алгоритмы Data Science и их практическая реализация на Python [Электронный ресурс]:учебное пособие. - Вологда: Инфра-Инженерия, 2022. - 392 с. – Режим доступа: https://e.lanbook.com/book/281189.
3. Криволапов С.Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python [Электронный ресурс]:Учебное пособие. - Москва: ООО "Научно-издательский центр ИНФРА-М", 2025. - 678 с. – Режим доступа: https://znanium.ru/catalog/document?id=450872.
4. Коротеев М.В. Основы машинного обучения на Python [Электронный ресурс]:Учебник. - Москва: КноРус, 2025. - 431 с. – Режим доступа: https://book.ru/book/957785.
5. Груздев А. В. План, примеры и метрики качества [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 814 с. – Режим доступа: https://e.lanbook.com/book/314948.
6. Груздев А. В. Инструменты и валидация [Электронный ресурс]:. - Москва: ДМК Пресс, 2023. - 816 с. – Режим доступа: https://e.lanbook.com/book/314945.